Software Engineering

4th Sem

Presented by Ankita Prasad 2026

WHAT IS SOFTWARE ?

Software is a program or set of programs containing instructions that provide the desired functionality.

Types of Software :

Software is generally classified into four major categories:

1. System Software

System software manages and controls the hardware and provides a platform for application programs to run.

Purpose :

Operates the computer

Controls memory, processor, and devices

Acts as an interface between hardware and user.

Real Examples :

Operating Systems (Windows, Linux, Android)

Device Drivers (printer driver, graphics driver)

Utility programs (disk cleanup, antivirus)

Example in real life :

When a student turns on a computer in a lab, the operating system loads first and allows all other programs to run. Without it, the computer cannot function.

2. Application Software

Application software is designed to help users perform specific tasks.

Purpose :

- Solves user problems
- Used in daily activities

Real Examples :

- Word processors (typing assignments)
- Web browsers (searching study material)
- Banking applications
- Online class platforms

Example in real life :

A student prepares an assignment using a word processing application and submits it online.

3. Embedded Software

Embedded software is built into hardware devices and controls their operation.

Purpose :

- Works inside machines
- Performs dedicated functions.

Real Examples :

- Washing machine controller
- Microwave oven timer
- ATM machine software
- Car braking system

Example in real life:

When a user selects “2 minutes” in a microwave oven, the internal software controls heating automatically.

4. Web Application Software

Web applications run on a server and are accessed through a web browser over the Internet.

Purpose :

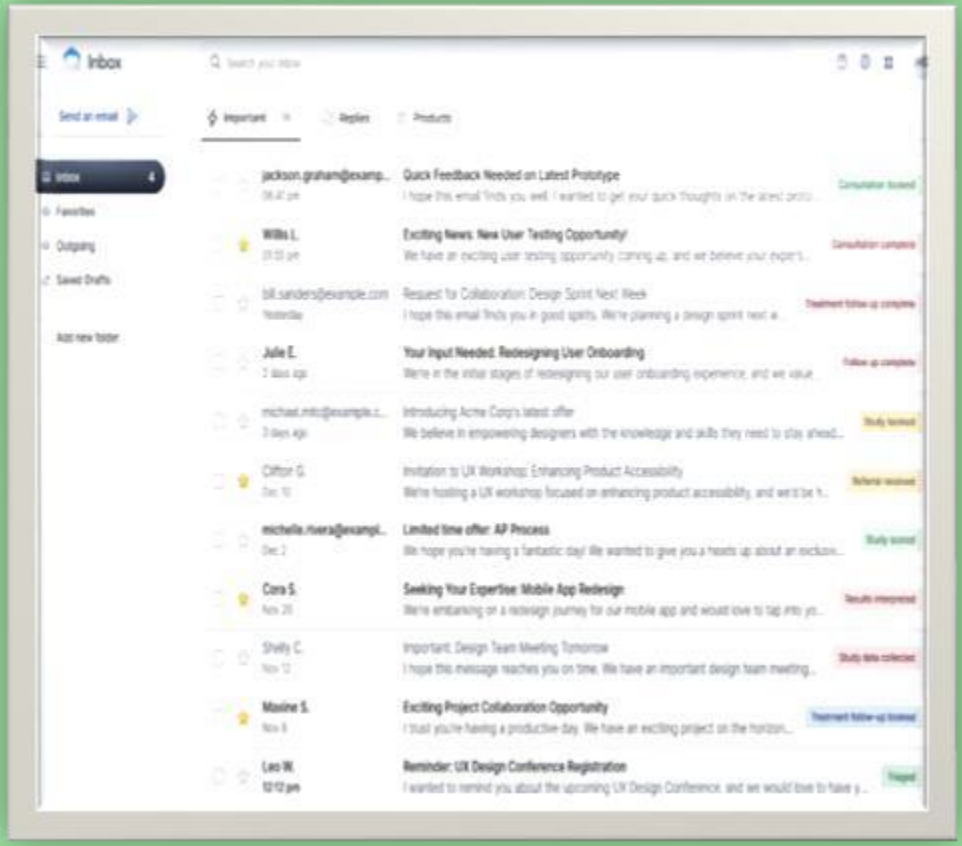
- Online services
- Accessible from anywhere

Real Examples :

- Online shopping websites
- Email services
- Online result portals
- Learning management systems

Example in real life :

Students check examination results using a college portal through a browser without installing any software.



CONCLUSION

Different types of software work together to make modern computer systems useful and user-friendly.

What is Software Engineering?

Software Engineering is the application of engineering principles to the design, development, testing, deployment, and maintenance of software.

IEEE Definition:

Software Engineering is the systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software.

EXPLANATION OF SOFTWARE ENGINEERING :

1.Application of Engineering :

Using engineering principles, methods, and techniques while developing software (just like civil engineers follow rules to construct a bridge).

2. Systematic :

Developing software in a proper step-by-step manner with planning.

- First requirements
- Then design
- Then coding
- Then testing

"Not done randomly"

3. Disciplined :

Following defined rules, standards, and processes during development.

- Time discipline
- Proper documentation
- Team coordination

4. Quantifiable :

Everything should be measurable.

- How much cost occurred
- How much time was required
- How many errors exist

" Decisions are based on numbers, not guesses"

5. Development :

Creating or building the software.

6. Operation :

The software is actually used and run by users.

7. Maintenance :

After release, the software is:

- Updated
- Bugs are fixed
- New features are added

"Software Engineering = Planning + Discipline + Measurement + Quality Software"

Need of Software Engineering :

- To manage complexity
- To reduce cost and time
- To improve quality
- To meet user requirements
- Thinking improve
- Interview
- Team work and Documentation
- Real project work

Software Engineering is required because modern software systems are large, complex and critical to daily life.

Without proper engineering practices, software projects may suffer from:

- **Delay in completion**
- **Increased cost**
- **Poor reliability**
- **Security problems**
- **System failure**

Software Engineering provides planning, standards, testing and documentation to ensure dependable software.

Career in Software Engineering :

After studying Software Engineering, students can work in the following roles:

- **Software Developer**
- **Web Developer**
- **Mobile Application Developer**
- **Software Tester / QA Engineer**
- **System Analyst**
- **DevOps Engineer**
- **Database Engineer**
- **Software Architect**

Software Engineering is one of the most demanded careers in IT industries, startups, banking systems, healthcare systems and government projects.

Why Study This Subject?

This subject teaches how to:

- Build software properly (not just coding)
- Work in a team
- Handle real industry projects
- Avoid software failures
- Improve software quality

Question:
Name any software that was popular earlier but is no longer used today.

- Orkut (social networking service)
- Adobe Flash Player
- Old media players or unsupported operating systems

Why did these software systems fail?
(Lack of updates, security issues, changing user needs)

Importance of Software in Daily Life :

Software has become an essential part of modern life. Almost every activity today depends on software systems for speed, accuracy, and convenience.

1. Communication :

Software allows people to communicate instantly across the world.

Examples:

- Messaging and video calls
- Email services
- Social media platforms

Without software, long-distance communication would be slow and difficult.

2.Education :

Learning is now highly dependent on software tools.

Uses:

- Online classes and lectures
- Digital notes and presentations
- Educational apps and simulations

"Students can learn anytime and anywhere."

3. Banking & Finance :

All financial transactions are handled by software systems.

Activities:

- Online banking
- ATM transactions
- UPI and digital payments
- Account management

Money transfer takes seconds instead of days.

4.Transportation & Navigation

Software helps in travelling safely and efficiently.

Examples:

- GPS navigation
- Ticket booking systems
- Traffic management systems

People reach destinations faster and avoid traffic.

5. Healthcare :

Hospitals depend heavily on software.

Uses:

- Patient records
- Medical reports
- Appointment scheduling
- Diagnosis machines

Improves treatment accuracy and saves lives.

6.Entertainment :

Software provides relaxation and recreation.

Examples:

- Music and video streaming
- Games
- Movies

Entertainment is available anytime on any device.

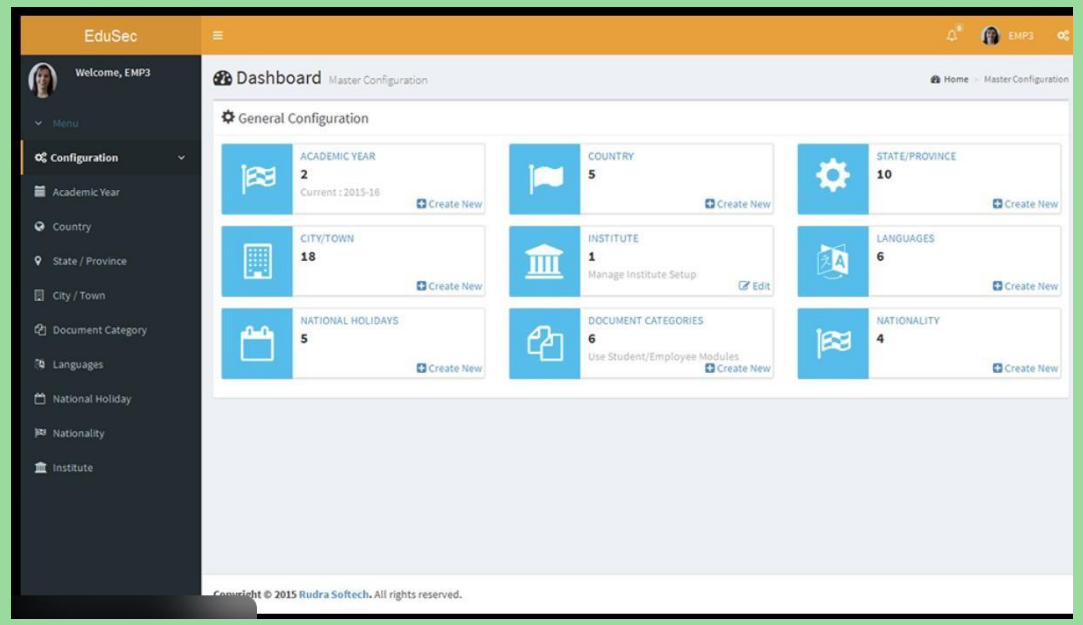
Conclusion

Software makes daily life faster, easier, more accurate, and more connected.
Without software, modern society would almost stop functioning.

Characteristics of Software

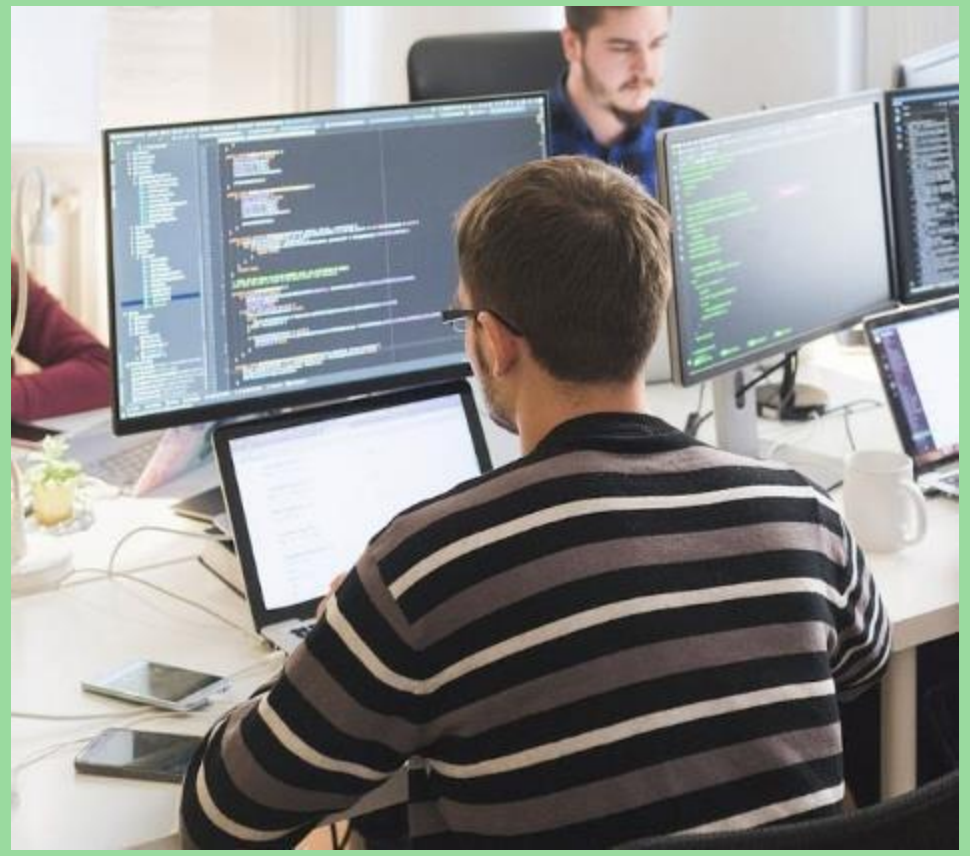
1.Most software is Custom-Built : Software developed for a specific organization or user requirement.

Real Example: College Management System made only for one institute.



2.Software is Developed or Engineered, It is Not Manufactured :
Software is engineered through design and coding, not mass-produced like physical products.

Real Example: A programmer writes code for an app — it is not produced in a factory like a keyboard.

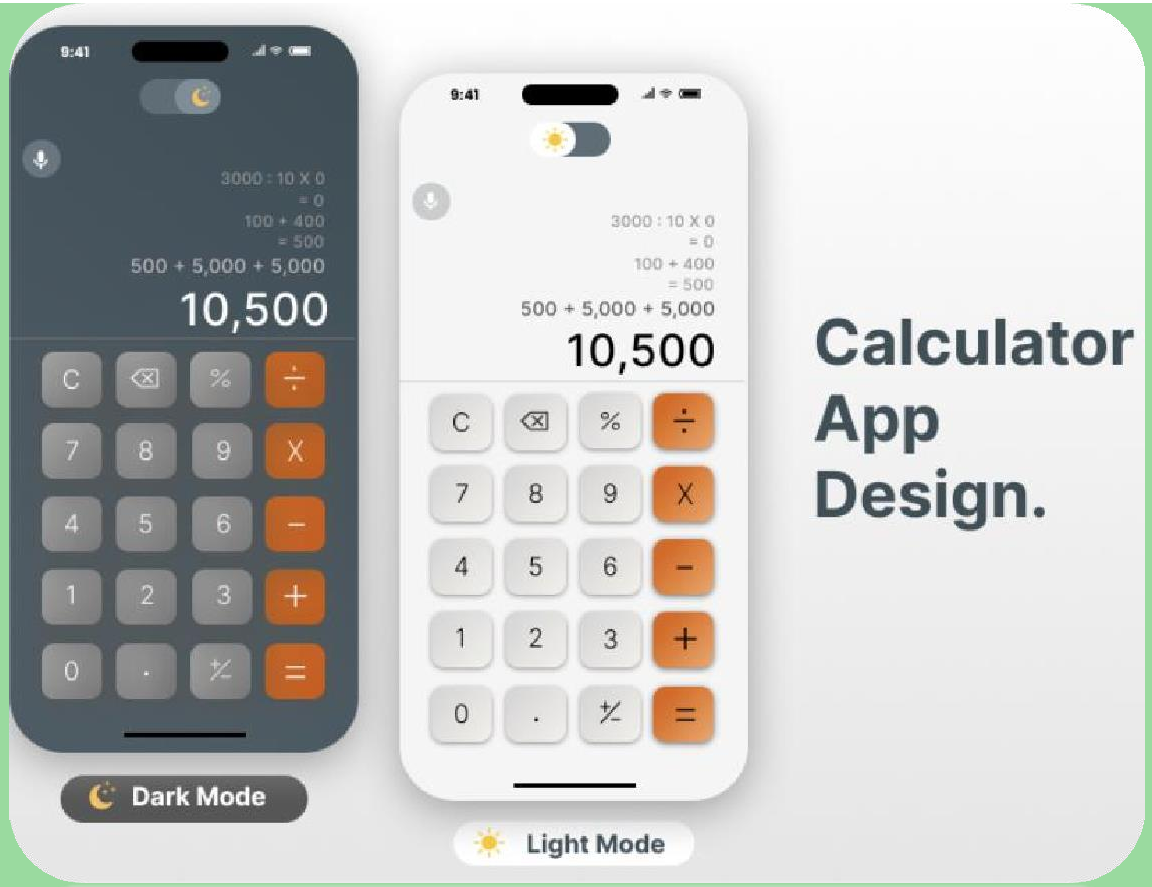


3.Software is Flexible Software :
Software can be easily modified or updated to meet changing requirements.

Real Example: A payment app adds UPI, QR payment, and new features through updates.

4.Software Doesn’t Wear Out :
Software does not physically degrade over time; failures occur due to bugs or environment changes.

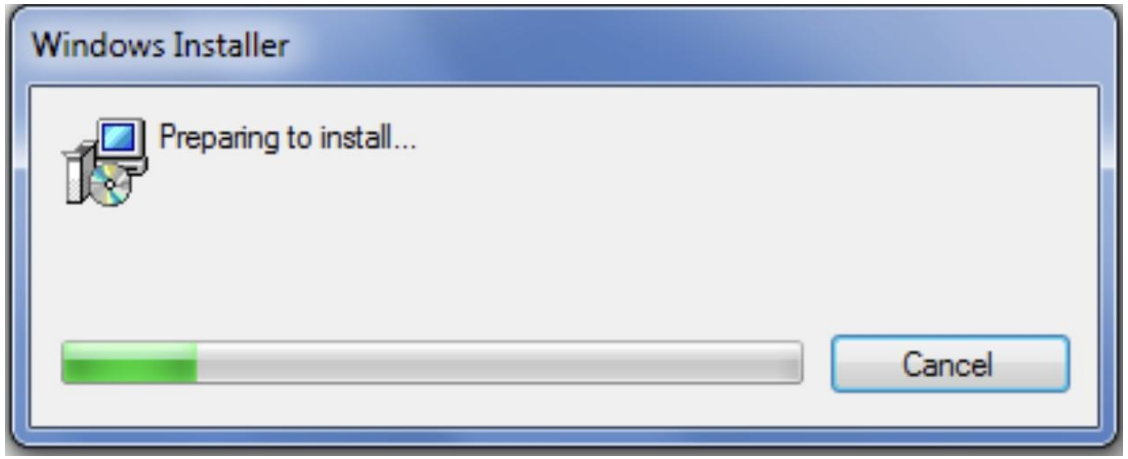
Real Example: A calculator app works same after 5 years if system supports it.



5.Becomes Obsolete with Time :
Software becomes outdated when new technology replaces it.

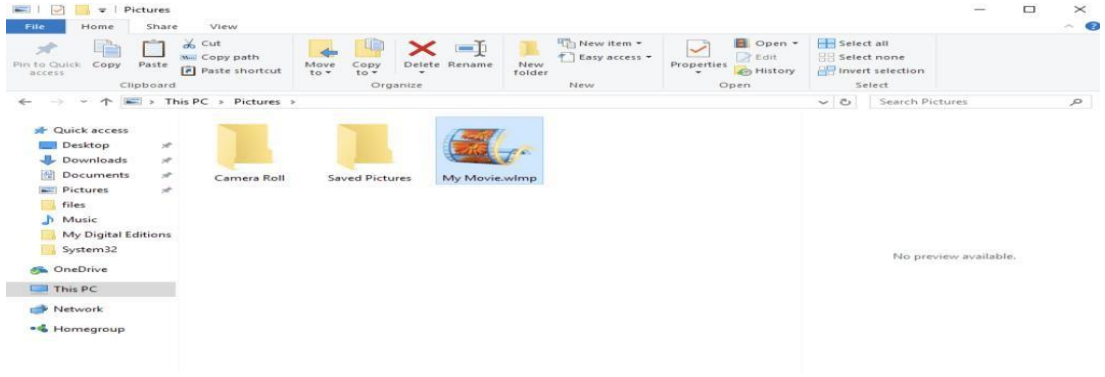
Real Example: Old media players or unsupported operating systems replaced by modern apps.

6.Software needs Maintenances.



"Software requires regular maintenance to fix errors and improve performance."

7.Software is easy to copy.



"Software can be duplicated easily without physical manufacturing."

Software Principles :

SE Principles are **basic guidelines** that help in developing **high-quality, reliable, and maintainable software**.

Important Software Principles :

1. Abstraction :

- Focus on **what** the system does, not **how** it is done.
- Helps reduce complexity.

Example :

User uses ATM, internal processing is hidden.

2. Modularity :

- Divide Software into small , manageable modules. each module performs a specific tasks.

Example :

Login , Payment, Report etc.

3. Information :

- Internal details of a module should be hidden from others.
- Only necessary information should be exposed.

Benefit:

Changes in one module do not affect others.

4. Low Coupling :

- Modules should have **minimum dependency** on each other.

Benefit:

Easy to modify and test.

5. High Cohesion :

- Each module should do **only one well-defined task**.

Benefit:

Better readability and maintainability.

6. Reusability :

- Software components should be reusable in other projects.

Example:

Login or authentication module reused in many apps.

7. Simplicity :

- Design should be **simple and easy to understand**.
- Avoid unnecessary complexity.

Rule:

Simple design = fewer bugs

8. Maintainability :

- Software should be easy to update, fix, and improve.

Example:

Bug fixing, adding new features.

Software Components :

A Software Components Is A System Element Offering A Predefined Services And Able To Communicate With Other Components

Clemens Szyperski and David Messerschmitt give the following five criteria for what a software components shall be to fullfill the definition :

1. Mutiple use
2. non – context – specific
3. Composable with other components
4. Encapsulated
5. Unit of Independent Deployment deployment & versioning :

Software Crisis

The term **software crisis** was introduced in the late 1960s - 1980s when organizations started building large and complex programs.

Projects frequently failed — they were late, over-budget, unreliable, or completely unusable.

In simple words,

people knew how to program, but they did not know how to manage software development.

The crisis was not lack of computers — it was lack of proper process.

What is Software Crisis?

Software Crisis is the set of problems faced during software development such as delay ,high cost, low quality and project failure.

Why did the Software Crisis happen?	
Increasing complexity	Programs became too large for individuals to handle
No development process	Coding started without planning
Poor communication	Users and developers misunderstood requirements
No testing standards	Errors discovered after delivery
No maintenance planning	Software failed after deployment

"Poor planning and complexity in large projects led to the Software Crisis."

When did it occur?

- Late **1960s – 1980s**
- After businesses and governments started depending on computers
- Demand for software grew faster than developers' ability to manage projects

What problems occurred?

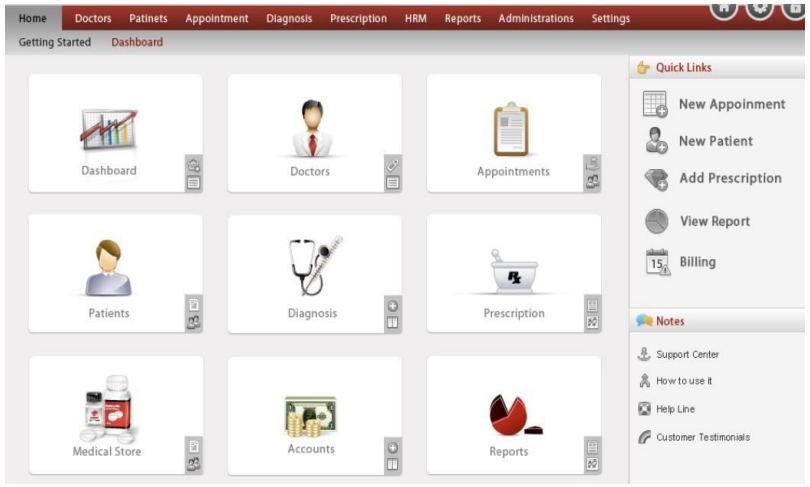
- Projects cancelled before completion
- Huge cost overruns
- Low quality software
- Frequent system crashes
- Unsafe systems in critical fields (medical ,banking)

Healthcare System Failure (Similar to Pandemic Management Systems)

What happened:
During large disease outbreaks, some countries initially failed to track infected patients because hospital databases were not integrated.

Why:
Different hospitals used different software formats. Systems were not designed to share data.

Result:
Delay in identifying patients and resource allocation.



Solution (Software Engineering approach):

- **Standardized databases :** Databases designed using common formats and rules so that different systems can store and understand data in the same way.

- **API-based communication :** Different software systems communicate with each other using predefined interfaces called APIs (Application Programming Interfaces).
  "One system requests data and another system sends it automatically."

- **Cloud-based centralized monitoring dashboards :** An online dashboard hosted on cloud servers that collects and displays real-time data from multiple locations in one place.

After applying structured architecture and requirements engineering, tracking improved dramatically.

COVID-19 pandemic exposed software crisis because healthcare systems could not share real-time data due to unstructured and incompatible software systems.

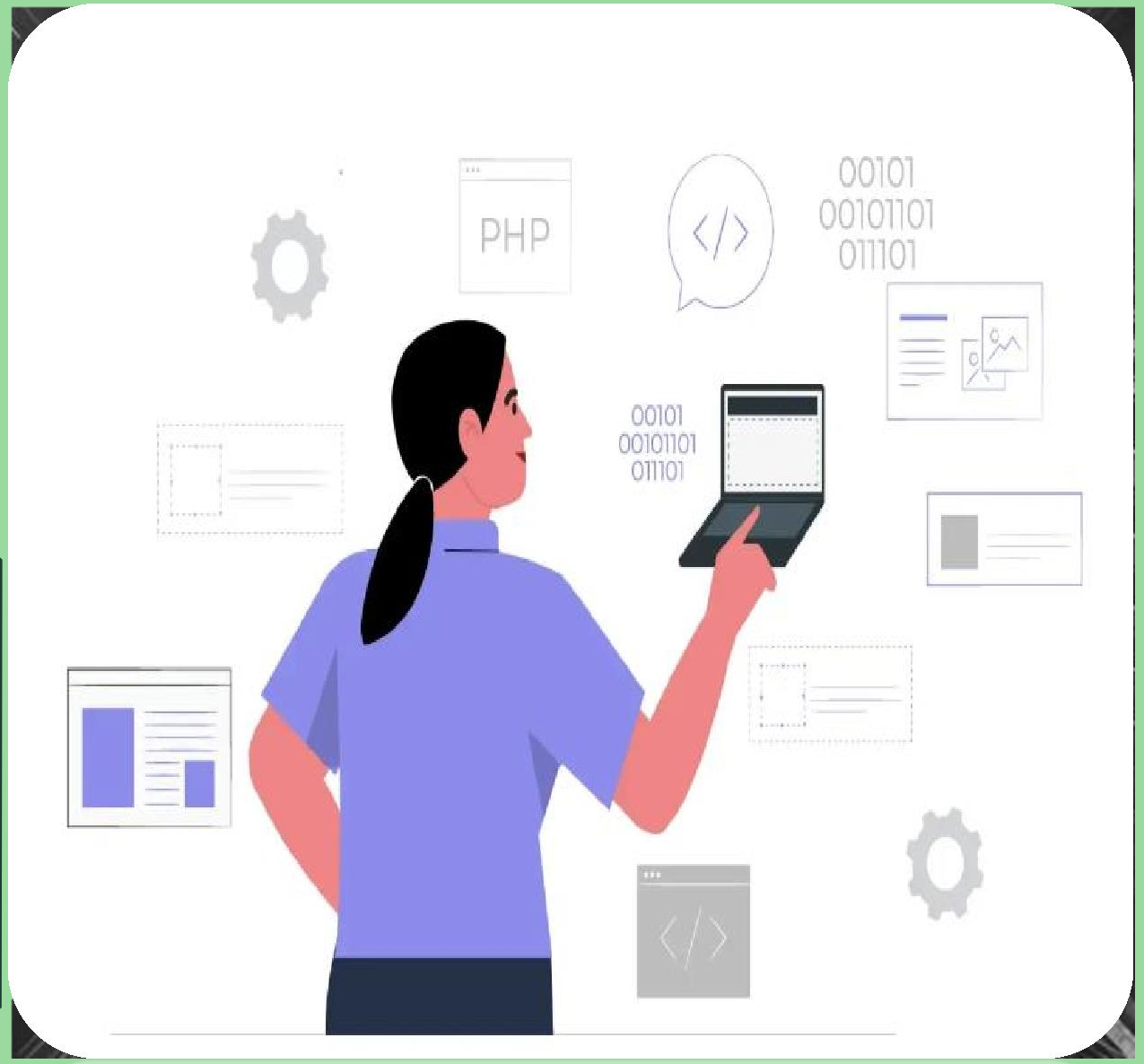
How Software Engineering Solved the Crisis ?

Crisis	Software Engineering Solution
Random coding	SDLC (step-by-step development)
Late error detection	Testing phases
Requirement confusion	Requirement analysis
Poor reliability	Quality assurance
Difficult maintenance	Modular design

Software Engineering introduced:

- Planning
- Documentation
- Design methods
- Testing standards
- Maintenance procedures

"So software development became an **engineering activity instead of trial-and-error programming**."



Conclusion

The software crisis proved that programming skill alone is not enough.

Reliable systems require structured processes, teamwork, and measurement — this led to the birth of **Software Engineering**.

Q. Name any ERP software used in hospitals worldwide.
Hospital Management System (HMS) / Electronic Health Record (EHR)

Q. Which Indian government app was used for COVID contact tracing?
Aarogya Setu

SOFTWARE ENGINEERING PROCESS

Process : A Process Is A Series Of Steps Involving Activities Constraints And Resources That Produce An Intended Output Of Some Kind.

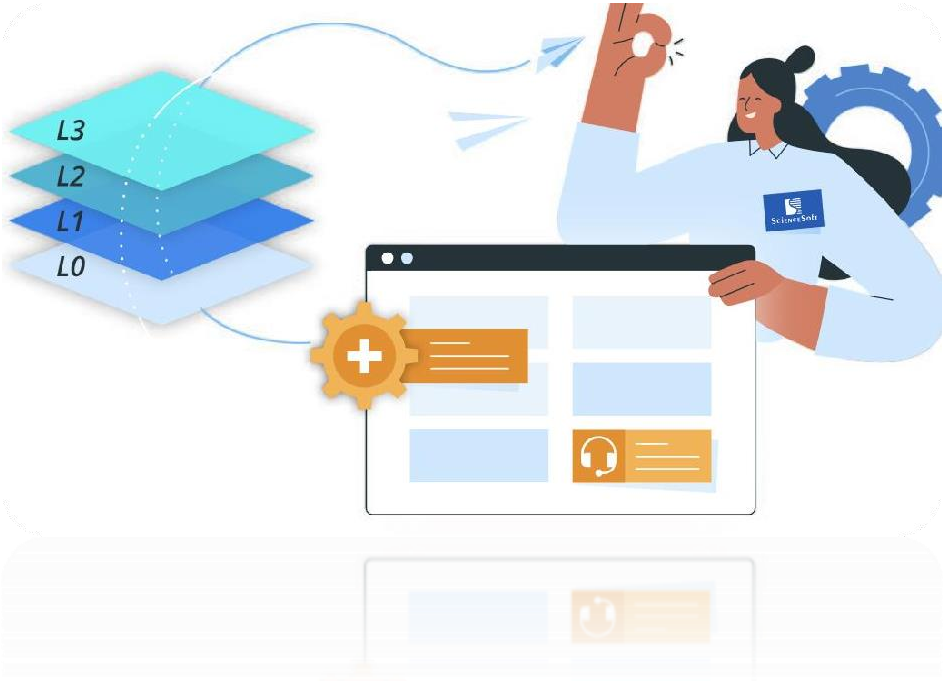
Input → Steps → Output

Example:
Raw data → processing → report

Making tea :

- Boil water
- Add tea leaves
- Add milk & sugar
- Serve

Steps follow an order → it is a *process*



Software Process : Software Process is the related set of activities and processes that are involved in developing and evolving a software system.

It tells **how software is built step-by-step**.

Main activities:

- **Requirement analysis** – What user needs
- **Design –** Planning the system
- **Coding** – Writing program
- **Testing –** Checking errors
- **Maintenance** – Updating after release

Main Activities of Software Process :

There are **four fundamental activities** in every software process.

1. **Software Specification :** Software specification is the process of **defining the requirements and constraints of the software system.**

It describes **what the system should do and how it should behave.**

Example

Before developing a **Hospital Management System**, developers collect requirements such as:

- Patient registration
- Doctor appointment system
- Medical records storage
- Billing system

These requirements are written in a document called **SRS (Software Requirement Specification).**

2.Software Development : Software development is the process of **designing and implementing the software according to the specified requirements.**

This phase includes:

- System design
- Coding
- Module creation

Example

Developers write programs for:

- Patient database system
- Online appointment booking
- Report generation

Languages used may include:

- Java
- Python
- PHP
- C#

3.Software Validation : Software validation ensures that the **developed software works correctly and meets the customer's requirements.**

This is done through **testing and verification.**

Example

For example, in an **Online Shopping App**:

- Check if payment gateway works correctly
- Verify if order confirmation is generated
- Ensure user login works properly

Types of testing:

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

4.Software Evolution : Software evolution refers to **modifying and updating software to meet changing customer requirements.**

Software must be updated regularly to remain useful.

Example

For example:

- WhatsApp added **video calling**
- Instagram added **Reels**
- Banking apps add **UPI payment features**

These updates are part of **software evolution.**

Types of Software Process Activities

According to the diagram in the book, **Software Process is divided into two main categories:**

```
                    Software Process
                          |
        ------------------------------------
        |                                  |
Product Engineering Processes   Process Management Processes
        |
-------------------------------------------------------------
   |                  |                          |
Development      Project Management      Software Configuration
Process              Process                     Management
```

Software Process Model : A **software process model** is a **structured framework** that defines **how software process activities are organized and performed**.

Examples of Models

- Waterfall Model
- Prototype Model
- Spiral Model
- Incremental Model
- Agile Model

Process = General steps for doing any work

Software Process = Steps specifically for making software

Software Process Model = Organized method/pattern for performing software process

Let's use Cooking real life example:

Process

Cooking food in general

Software Process

Cooking a **specific dish** (like paneer curry) with required kitchen tasks

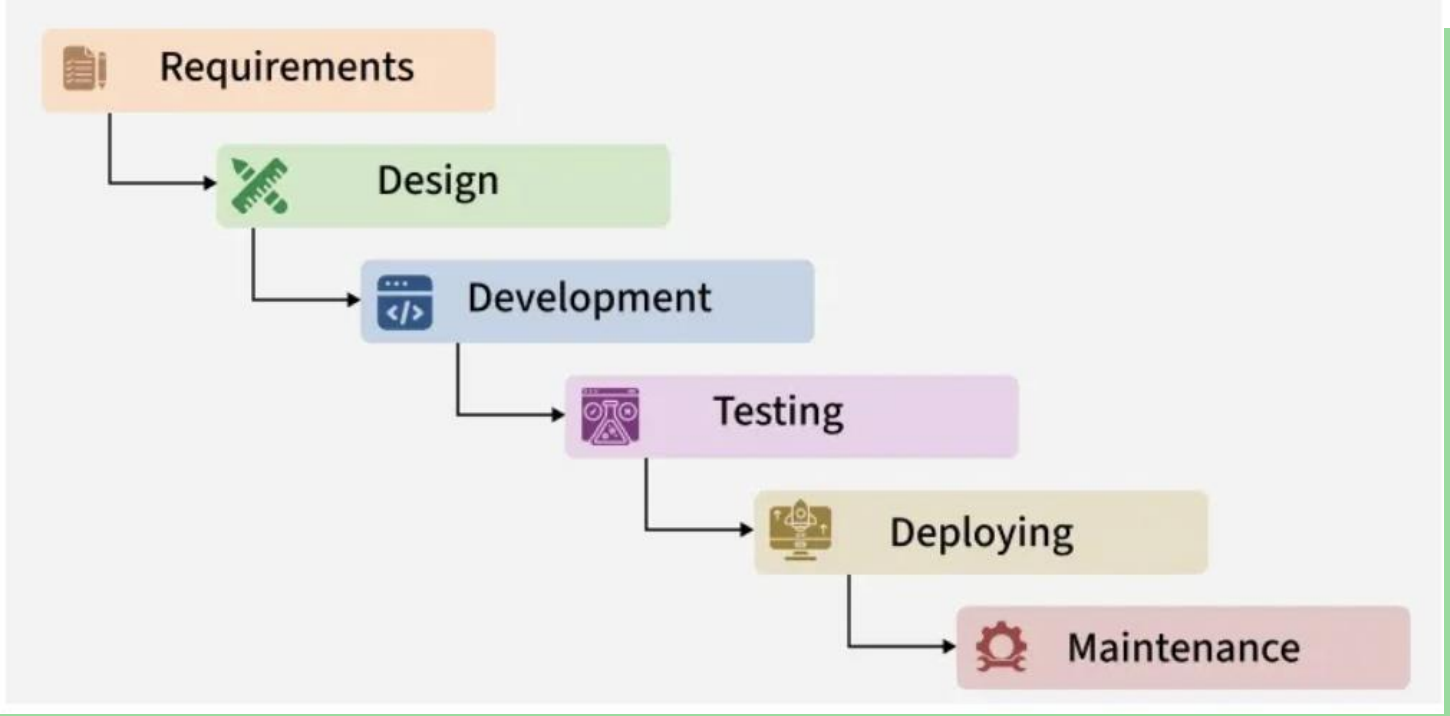
Software Process Model

The **recipe style** you follow:

- Step by step recipe
- Trial and improve recipe
- Make in parts recipe

Waterfall Model : The Waterfall Model is a traditional software development methodology that follows a linear, phase-by-phase approach, where each phase must be completed before moving to the next. It does not allow backtracking and permits only minimal changes once a phase is completed.

The model is simple, well-structured, and easy to manage, offering high predictability and clearly defined milestones throughout the development process.



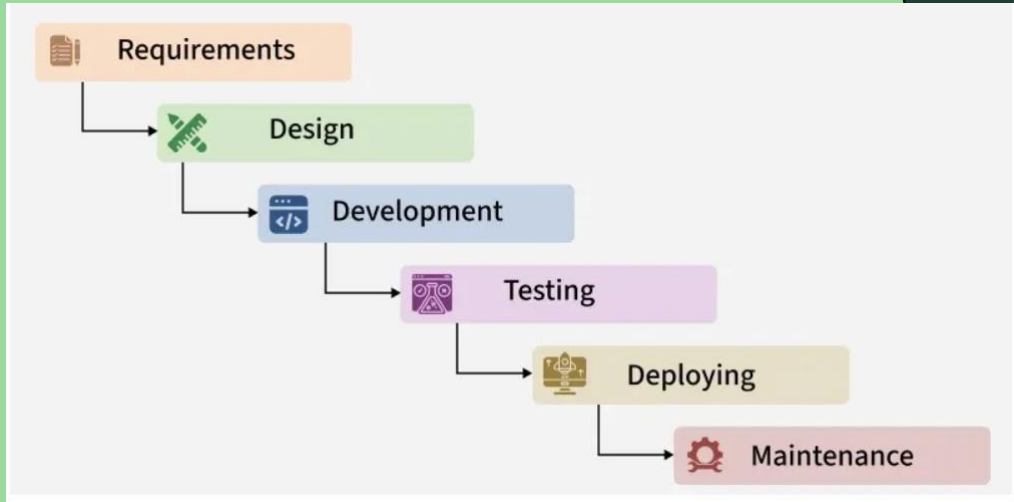
Features of the Waterfall Model

- **Sequential Approach**
  Development follows a linear, step-by-step process where each phase is completed before moving to the next.

- **Document-Driven Process**
  Detailed documentation is created at every stage to clearly define requirements, design, and progress.

- **Quality Control**
  Strong emphasis is placed on verification and testing to ensure each phase meets its defined requirements.

- **Detailed Planning**
  The project scope, schedule, and deliverables are carefully planned and monitored throughout the lifecycle.



1. Requirements Analysis and Specification

This phase focuses on clearly understanding and documenting the customer's needs.

- **Requirement Gathering and Analysis:**
  Customer requirements are collected and carefully examined to remove errors, confusion, and inconsistencies.
- **Requirement Specification:**
  The approved requirements are documented in the Software Requirement Specification (SRS), which serves as a formal agreement between the customer and the development team.

2. Design

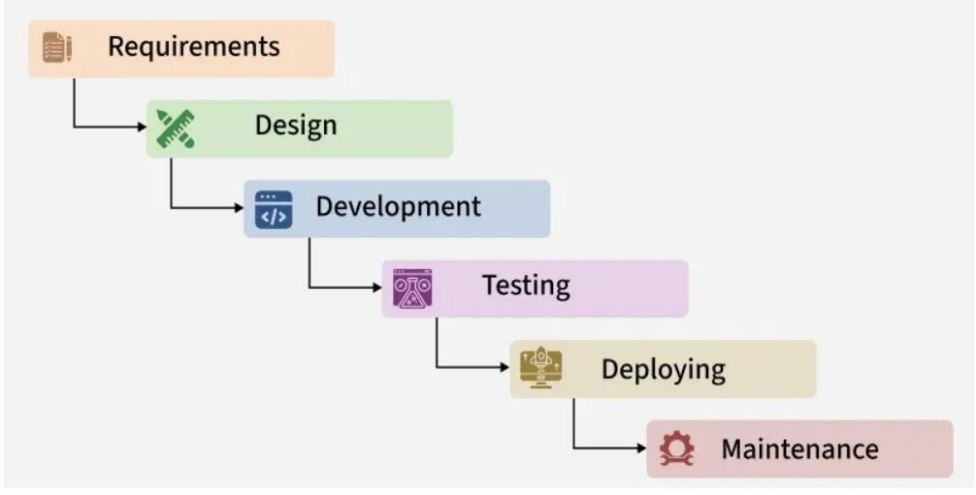
In this phase, the requirements from the SRS are converted into a system design that can be implemented in code.

- **High-Level Design (HLD):**
  Defines the overall system architecture, major components, and their interactions.
- **Low-Level Design (LLD):**
  Provides detailed design of each component, including logic and data flow, to guide developers during coding.
- All designs are documented in the Software Design Document (SDD).

3. Development

This phase involves converting the design into actual working software.

- Developers write source code based on the design documents.
- Suitable programming languages and tools are used.
- Unit testing is performed to verify that each module works correctly on its own.



4. Testing and Deployment

This phase ensures that the integrated software functions correctly and is successfully delivered for real world use.

Testing: After unit testing, software modules are integrated incrementally and tested at each stage. Once all modules are successfully integrated, full system testing is performed to ensure the system functions as expected.

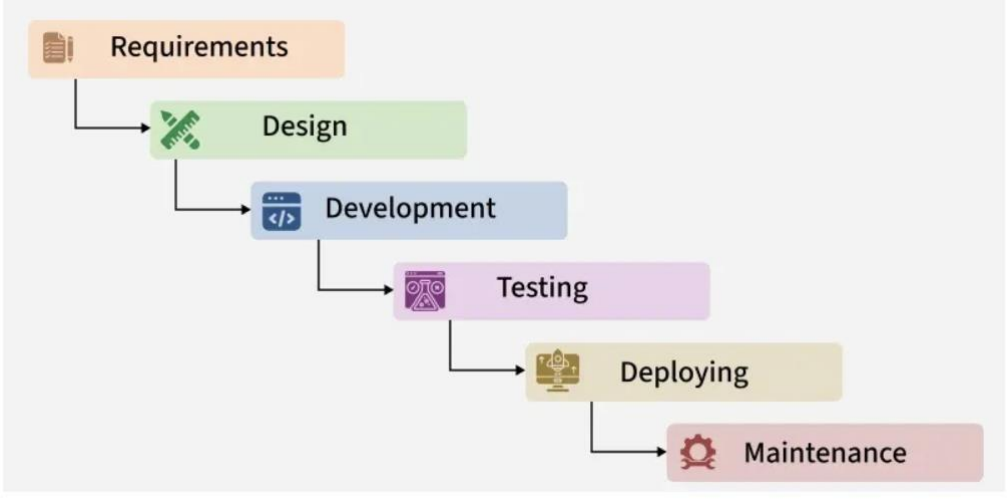
- **Alpha Testing:** Performed by the development team.
- **Beta Testing:** Performed by selected end users.
- **Acceptance Testing:** Performed by the customer to approve the software.

Deployment: After successful testing, the software is deployed to a live environment for end users. This phase includes environment setup, user training, and final checks to ensure smooth operation in real world conditions.

5. Maintenance

Maintenance ensures the software continues to function effectively after deployment.

- **Corrective Maintenance:** Fixes errors found after release.
- **Perfective Maintenance:** Enhances features based on user needs.
- **Adaptive Maintenance:** Updates software to work in new environments, such as new hardware or operating systems.



Advantages

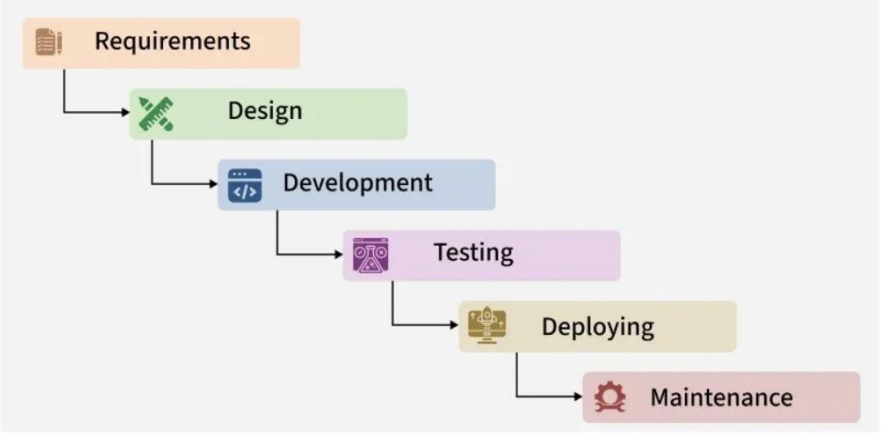
- **Easy to Understand:**
  The model is straightforward and simple, making it easy to learn and follow.

- **Sequential Processing:**
  Each phase is completed one at a time, ensuring an organized development flow.

- **Well-Defined Stages:**
  Every phase has clear objectives and deliverables, reducing confusion.

- **Clear Milestones:**
  Progress is easy to track due to clearly defined milestones.

- **Strong Documentation:**
  All processes, actions, and results are thoroughly documented.



- **Encourages Best Practices:**
  Promotes disciplined practices such as define before design and design before c*oding*.

- **Suitable for Small and Stable Projects:**
  Works well for smaller projects where requirements are clearly understood and unlikely to change.

When to Use Waterfall Model?

- **Well-Defined Requirements:**
  Requirements are clear, stable, and fully documented before development begins.

- **Minimal Changes Expected:**
  The project scope is unlikely to change during development.

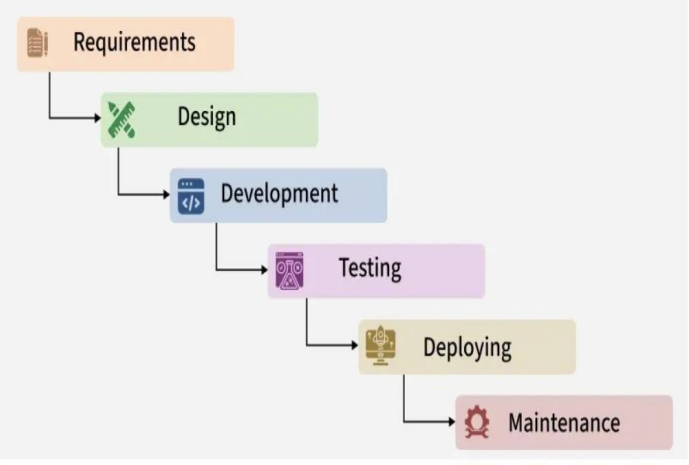
- **Small to Medium-Sized Projects:**
  Suitable for projects with limited complexity and a clear development path.

- **Predictable and Low Risk:**
  Risks are known, manageable, and can be addressed early in the development cycle.

- **Strict Regulatory Compliance:**
  Ideal when detailed documentation and adherence to standards or regulations are mandatory.

- **Client Prefers a Sequential Approach:**
  Best when the client wants a step-by-step, linear development process.

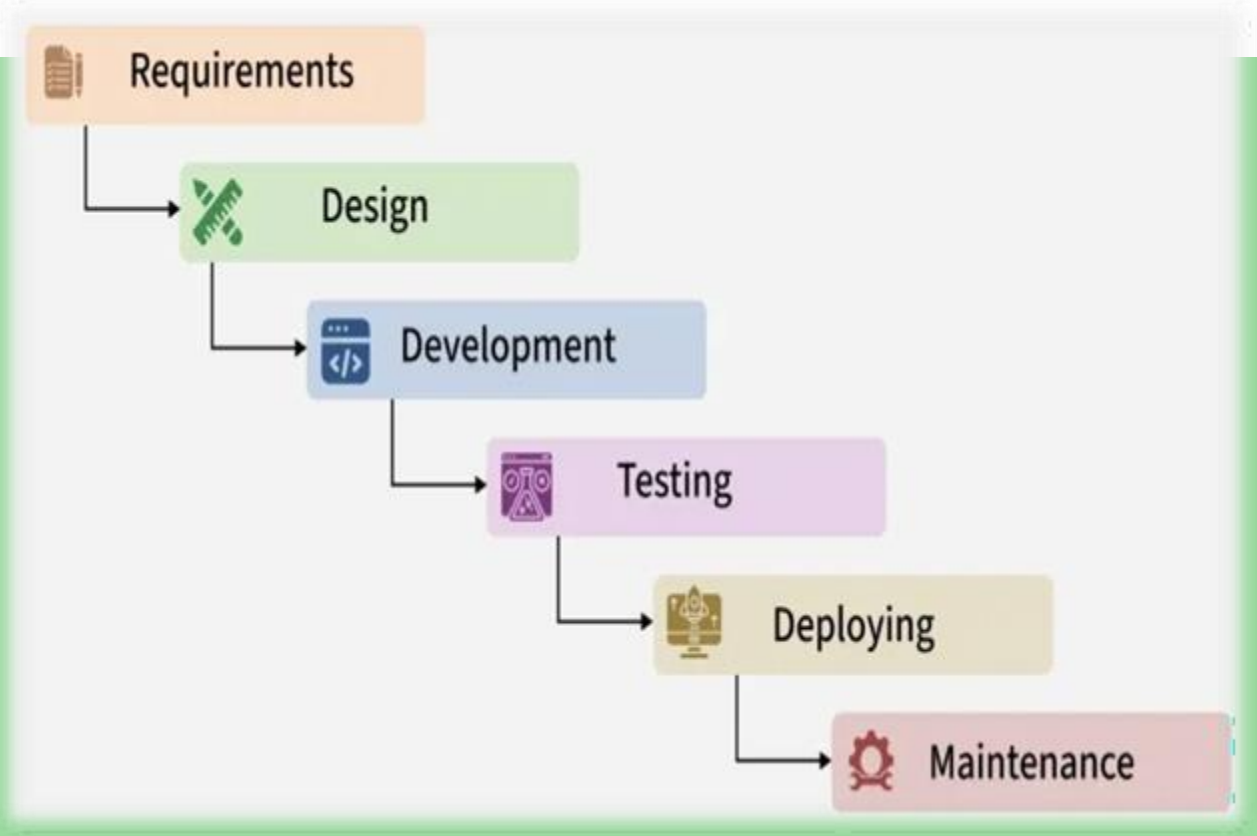
- **Limited Resources:**
  Works well when resources are limited and need careful planning and allocation.



Example of Waterfall Model

Below is a real-world example of the Waterfall Model using an **Online Food Delivery System**.

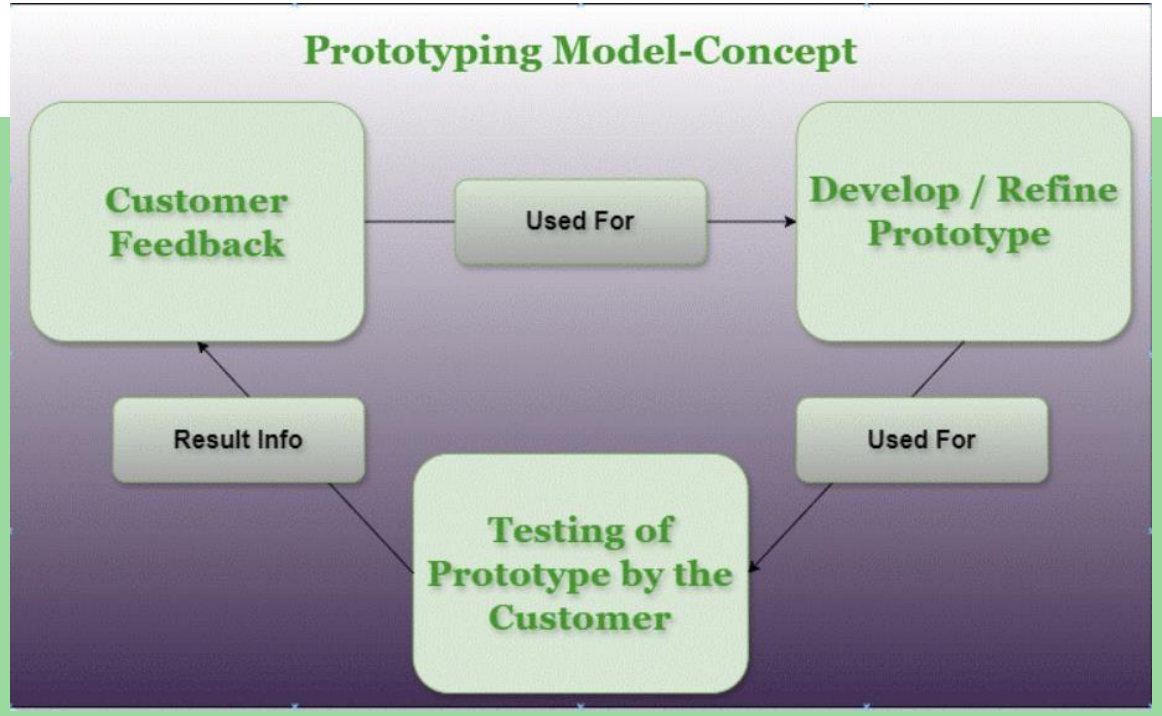
Why it is called Sequential Model?
It is fixed order.
Requirement → Design → Coding → Testing → Deployment



Prototyping Model :

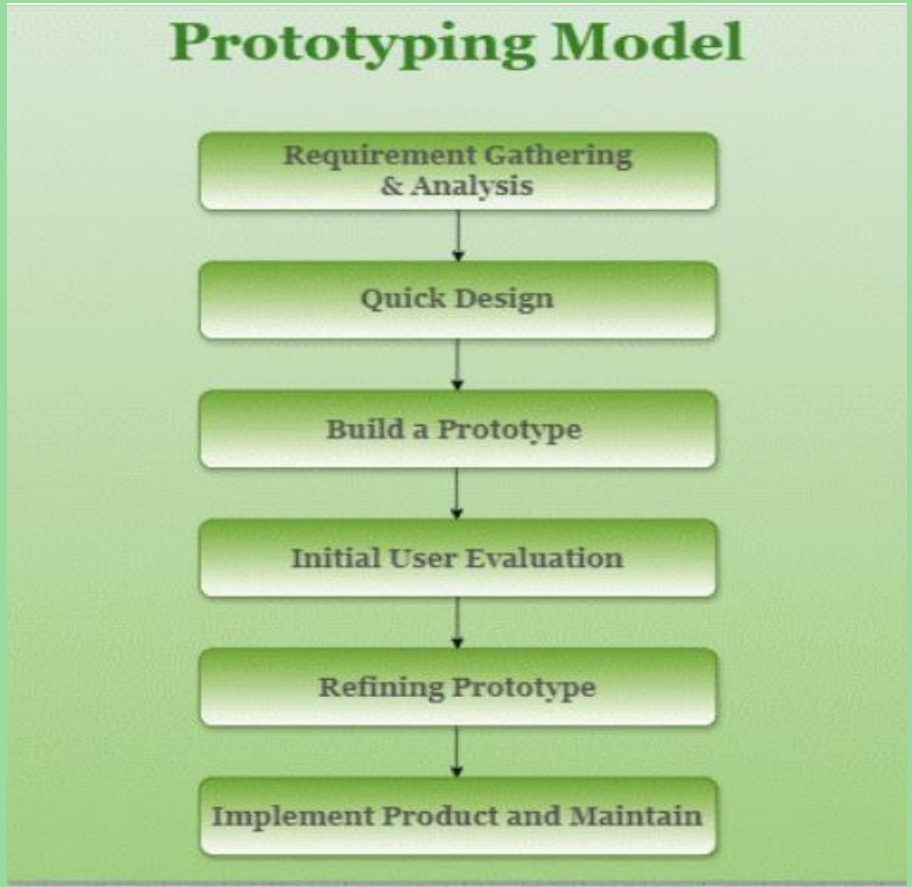
the Prototyping Model is one of the most popularly used Software Development Life Cycle Models (SDLC models). This model is used when the customers do not know the exact project requirements beforehand.

In this model, a prototype of the end product is first developed, tested, and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.



In this process model, the system is partially implemented before or during the analysis phase thereby allowing the customers to see the product early in the life cycle. The process starts by interviewing the customers and developing the incomplete high-level paper model.

This document is used to build the initial prototype supporting only the basic functionality as desired by the customer. Once the customer figures out the problems, the prototype is further refined to eliminate them. The process continues until the user approves the prototype and finds the working model to be satisfactory.



Prototyping Model Phases

Phases of Prototyping Model

Prototyping Model has six phases as follows:

1. Requirements gathering and analysis

Requirement analysis is the first step in developing a prototyping model. During this phase, the system's desires are precisely defined. During the method, system users are interviewed to determine what they expect from the system.

2. Quick design

The second phase could consist of a preliminary design or a quick design. During this stage, the system's basic design is formed. However, it is not a complete design. It provides the user with a quick overview of the system. The rapid design aids in the development of the prototype.

3. Build a Prototype

During this stage, an actual prototype is intended to support the knowledge gained from quick design. It is a small low-level working model of the desired system.

4. Initial user evaluation

The proposed system is presented to the client for preliminary testing at this stage. It is beneficial to investigate the performance model's strengths and weaknesses. Customer feedback and suggestions are gathered and forwarded to the developer.

5. Refining prototype

If the user is dissatisfied with the current model, you may want to improve the type that responds to user feedback and suggestions. When the user is satisfied with the upgraded model, a final system based on the approved final type is created.

6. Implement Product and Maintain

The final system was fully tested and distributed to production after it was developed to support the original version. To reduce downtime and prevent major failures, the programmer is run on a regular basis.

Types of Prototyping Models

There are four types of Prototyping Models, which are described below.

1. Rapid Throwaway Prototyping

- This technique offers a useful method of exploring ideas and getting customer feedback for each of them.
- In this method, a developed prototype need not necessarily be a part of the accepted prototype.
- Customer feedback helps prevent unnecessary design faults and hence, the final prototype developed is of better quality.

2. Evolutionary Prototyping

- In this method, the prototype developed initially is incrementally refined based on customer feedback till it finally gets accepted.
- In comparison to Rapid Throwaway Prototyping, it offers a better approach that saves time as well as effort.
- This is because developing a prototype from scratch for every iteration of the process can sometimes be very frustrating for the developers.

3. Incremental Prototyping

- In this type of incremental prototyping, the final expected product is broken into different small pieces ofprototypes and developed individually.
- In the end, when all individual pieces are properly developed, then the different prototypes are collectivelymerged into a single final product in their predefined order.
- It's a very efficient approach that reduces the complexity of the development process, where the goal is dividedinto sub-parts and each sub-part is developed individually.
- The time interval between the project's beginning and final delivery is substantially reduced because all parts ofthe system are prototyped and tested simultaneously.
- Of course, there might be the possibility that the pieces just do not fit together due to some lack of ness in thedevelopment phase – this can only be fixed by careful and complete plotting of the entire system beforeprototyping starts.

4. Extreme Prototyping

This method is mainly used for web development. It consists of three sequential independent phases:

- In this phase, a basic prototype with all the existing static pages is presented in HTML format.
- In the 2nd phase, Functional screens are made with a simulated data process using a prototype services layer.
- This is the final step where all the services are implemented and associated with the final prototype.

Advantages of Prototyping Model

- The customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.
- New requirements can be easily accommodated as there is scope for refinement.
- Missing functionalities can be easily figured out.
- Errors can be detected much earlier thereby saving a lot of effort and cost, besides enhancing the quality of the software.
- The developed prototype can be reused by the developer for more complicated projects in the future.
- Flexibility in design.

Disadvantages of the Prototyping Model

- Costly concerning time as well as money.
- There may be too much variation in requirements each time the prototype is evaluated by the customer.
- Poor Documentation due to continuously changing customer requirements.
- It is very difficult for developers to accommodate all the changes demanded by the customer.
- There is uncertainty in determining the number of iterations that would be required before the prototype is finally accepted by the customer.
- After seeing an early prototype, the customers sometimes demand the actual product to be delivered soon.
- Developers in a hurry to build prototypes may end up with sub-optimal solutions.

Spiral Model :

The Spiral Model is a risk-driven and iterative software development model. Development progresses in a spiral shape consisting of multiple loops, where each loop represents a complete development cycle.
The number of loops depends on project size, complexity, and risk.

Each loop includes:

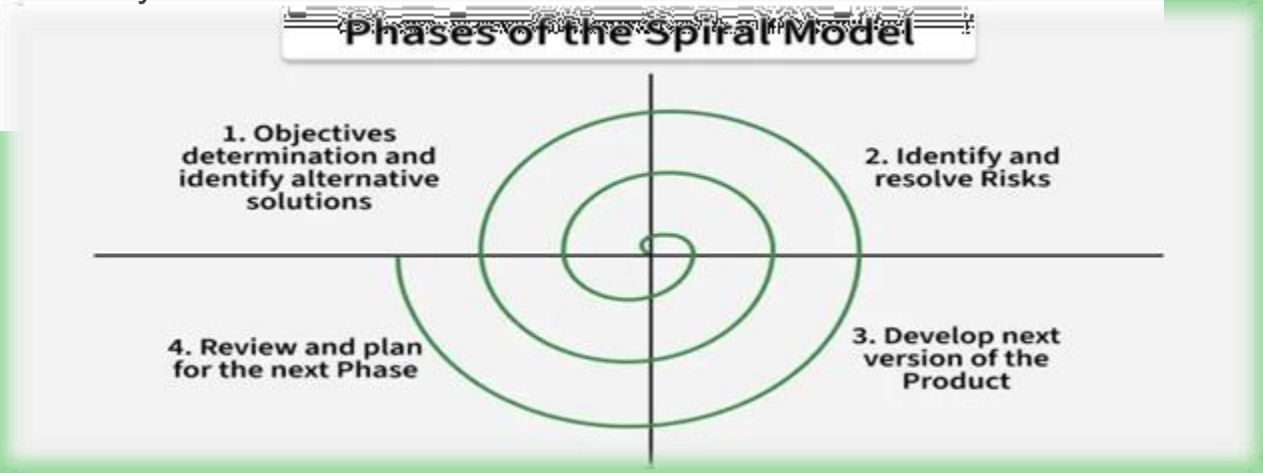
- Planning
- Risk analysis
- Development
- Evaluation

Phases of the Spiral Model

Focus is on managing risk through multiple iterations of the software development process. Each phase of the Spiral Model is divided into four Quadrants.

1. Objectives Definition

- Project goals and requirements are identified
- Functional and non-functional requirements are analyzed
- Possible solutions are explored



2. **Risk Analysis and Resolution**
- Risks related to cost, schedule, performance, and technology are identified
- Best solution is selected
- Prototypes are built to reduce risks

3. **Development and Testing**
- Selected features are designed, developed, and tested
- The next working version of the software is created

4. **Review and Planning**
- Customers evaluate the current version
- Feedback is collected
- Planning for the next iteration begins

Risk Handling
- Risks are identified after development starts
- Prototypes help detect and resolve risks early
- Continuous risk evaluation at every iteration
- More flexible than Prototyping and Waterfall models

*The Spiral model is called a **Meta-Model** because it subsumes all the other SDLC models.*

Example of Spiral Model (E-Commerce Website)

1. First Spiral - Planning and Requirements:
Basic requirements like product listing, cart, and payment are collected. Risks such as security and scalability are identified, and a simple prototype is built.

2. Second Spiral - Risk Analysis and Refining the Design
Feedback from the prototype is used to improve design. Secure payments, user registration, and cart features are added and tested.

3. Third Spiral - Detailed Implementation
Advanced features like order tracking, reviews, and search are developed. Scalability and performance risks are tested under heavy load.

4. Final Spiral - Deployment
The website is fully tested, launched for users, and monitored for remaining risks and improvements.
This example shows how the Spiral Model supports continuous improvement and risk management at every stage.

Advantages

- **Effective Risk Handling:** Identifies and resolves risks at every phase, making it ideal for projects with high uncertainty.
- **Suitable for Large Projects:** Best suited for large and complex software systems.
- **Flexible Requirements:** Allows changes in requirements even at later stages of development.
- **Early Customer Feedback:** Customers can review early versions of the product, improving satisfaction.
- **Iterative Development:** Software is developed in repeated cycles, allowing gradual improvement.
- **Better Communication:** Regular reviews improve coordination between customers and developers.
- **High Software Quality:** Continuous testing and refinement lead to reliable and high-quality software.

Disadvantages

- **Complex to Manage:** The model is complex due to multiple iterations and continuous risk analysis.
- **High Cost:** Not suitable for small projects because it requires more time, effort, and resources.
- **Depends Heavily on Risk Analysis:** Success depends on accurate risk identification, which requires experienced professionals.
- **Difficult Time Estimation:** The number of iterations is not fixed, making scheduling difficult.
- **Time-Consuming:** Frequent reviews and evaluations can increase development time.
- **Resource Intensive:** Requires significant investment in planning, risk management, and testing.

When To Use the Spiral Model?

- Large and high-risk projects
- Unclear or changing requirements
- Frequent releases required
- Prototyping is needed
- Risk and cost evaluation is critical
- Long-term projects with uncertainty

Incremental Process Model

Software is not delivered all at once. Instead, development is divided into multiple increments, and each increment goes through requirements, design, development, testing, and implementation phases. Every new release improves the previous version.

- **Step-by-Step Development:** The system is built gradually in small parts.
- **Early Working Software:** A usable version is delivered early.
- **Customer Feedback:** Feedback after each increment improves future releases.
- **Flexible to Changes:** New features can be added in later increments.
- **Structured yet Iterative:** Combines planning of Waterfall with flexibility of iteration.

Phases of Incremental Model

1. Requirement Analysis
Requirements for the current increment are gathered and clearly defined.

2. Design & Development
The system design is prepared, and new features are developed.

3. Testing
Each increment is tested to ensure new and existing features work correctly.

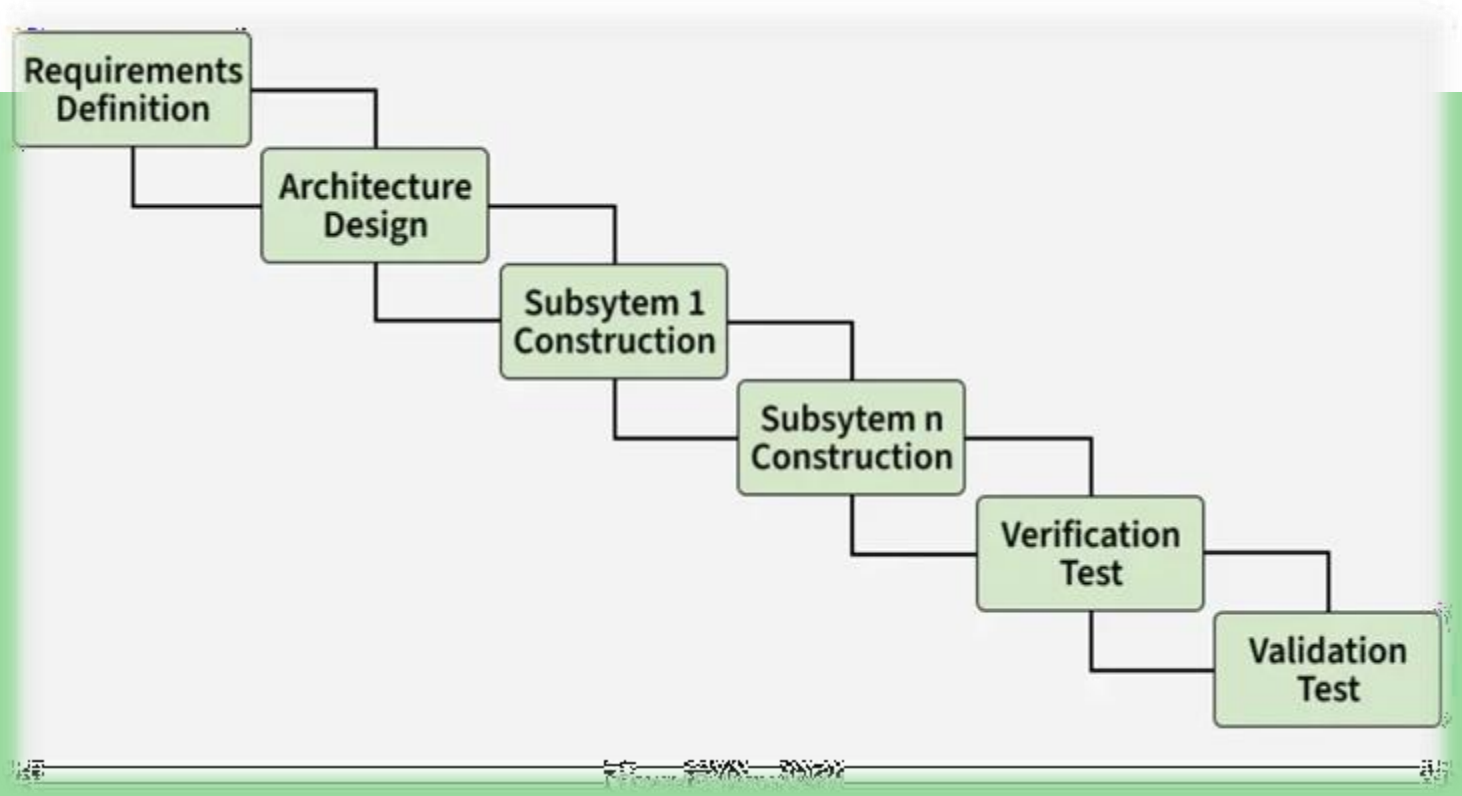
4. Implementation
The tested increment is delivered and integrated with previous versions.
These steps repeat until the final system is completed.

Types of Incremental Model

The Incremental Model has two main types, each offers different approaches to how software is developed in parts.

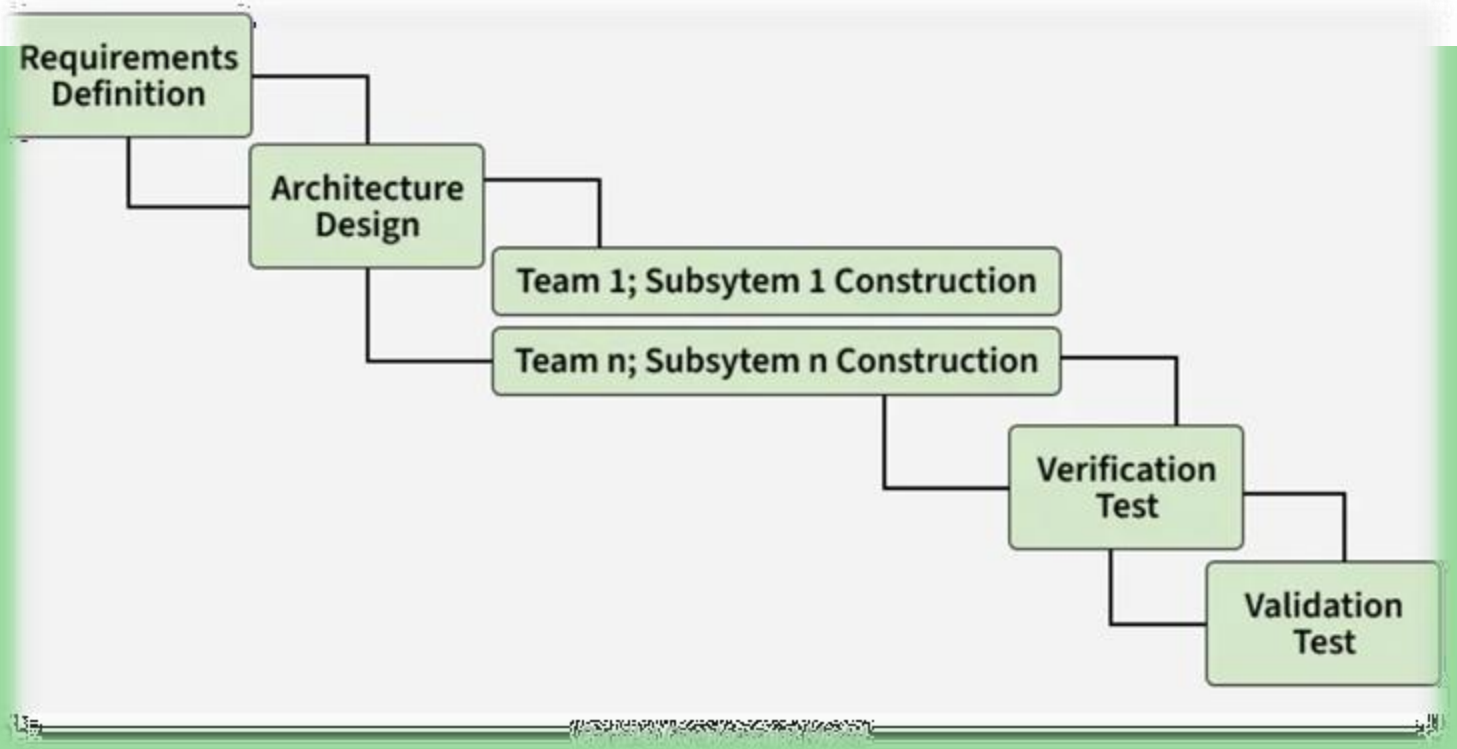
Staged Delivery Model

Develops software in a sequence of planned stages, where each stage delivers a functional part of the system. Each release brings the product closer to completion, allowing it to evolve gradually. Working versions are delivered at regular intervals, making progress visible and manageable throughout the development process. The diagram below shows this model :



Parallel Development Model

Divides the system into multiple modules that are developed simultaneously at the same time by different teams. By working on separate components in parallel, the development process becomes faster and more efficient. This approach reduces overall project time and allows teams to focus on specific functionalities concurrently. Given below is the diagram showing the model:



Use Cases of Incremental Process Model

- **When the requirements are well-defined and clear**
  Because increments can be planned and developed step-by-step with minimal requirement changes.

- **If the project has a long development timeline**
  Incremental delivery helps manage complexity over time by breaking the project into smaller, manageable parts.

- **If the customer needs a quick product release**
  You can deliver the most critical features early in the first increment, allowing the customer to start using the software sooner.

- **When you want to develop the most important features first**
  This allows early feedback on key functionalities and better prioritization for subsequent increments.

Note: The Model is not ideal when the software development team is not highly skilled or experienced because managing increments and integrating them well requires a competent team.

Advantages

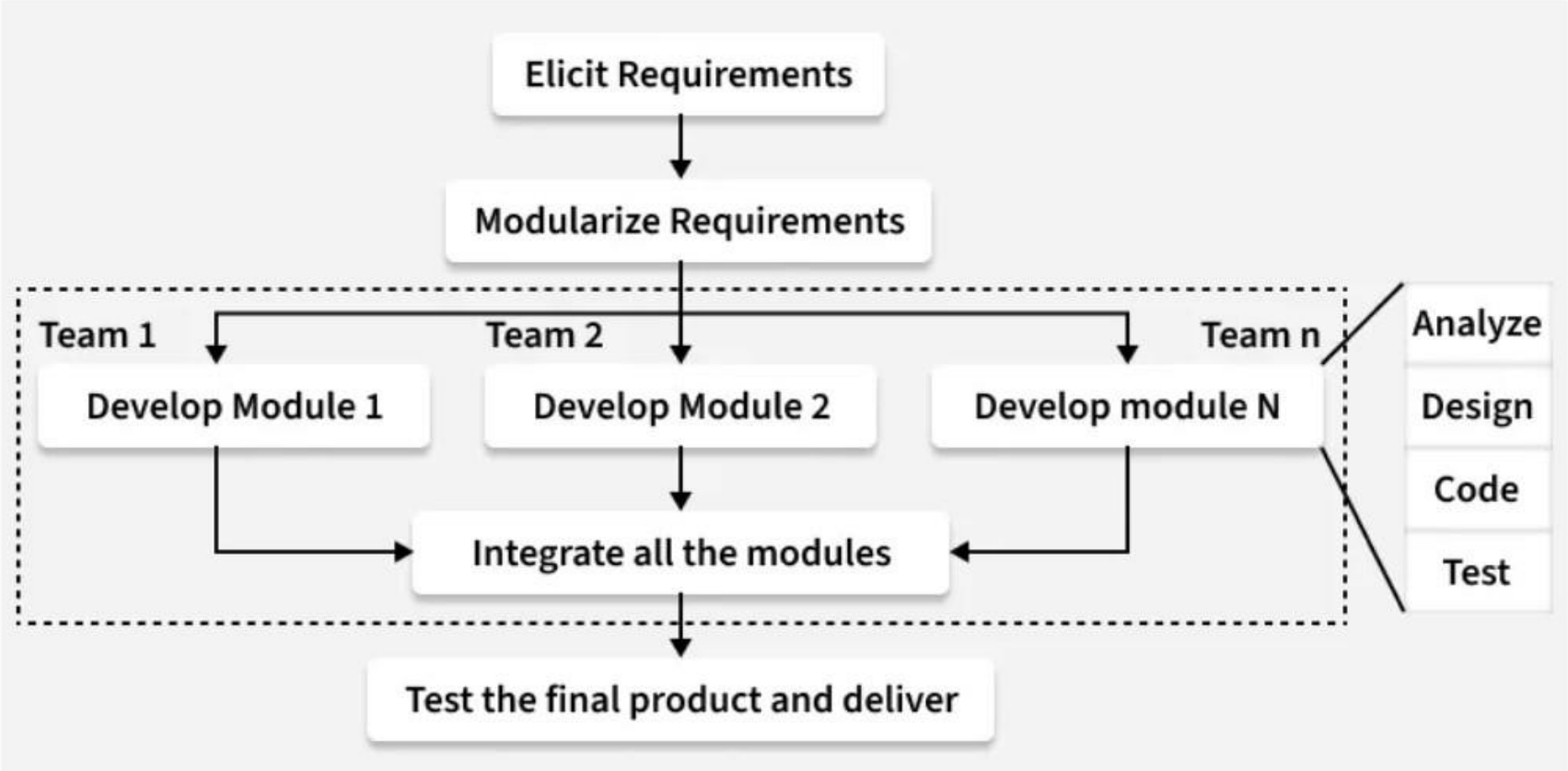
- Early Software Delivery, improves customer satisfaction
- Easy to Manage Changes between increments
- Better Risk Management through early testing
- Cost Control by spreading budget over stages
- Simpler Debugging due to modular development
- High Software Quality through continuous testing

Disadvantages

- Requires Skilled Team and proper planning
- Cost May Increase due to repeated testing and integration
- Design Issues if future requirements are unclear
- Integration Challenges between increments
- Repeated Bug Fixing across multiple increments

Rapid Application Development Model (RAD)

The RAD (Rapid Application Development) Model is a software development methodology that focuses on fast development and frequent releases of working software. It emphasizes iterative development, quick user feedback, and flexibility, allowing changes to be easily incorporated during the development process. Unlike traditional models such as the Waterfall Model, RAD adapts quickly to changing requirements and user needs.



RAD Model

RAD Model

The RAD (Rapid Application Development) Model is an incremental software development model proposed by IBM in the 1980s that focuses on fast development using short time-boxed cycles (usually 60–90 days). It divides the system into small modules developed in parallel using powerful tools, with continuous user feedback through prototypes.

The use of powerful developer tools such as JAVA, C++, Visual BASIC, XML, etc. is also an integral part of the projects. This model consists of 4 basic phases:

• **Requirements Planning** – In this phase, developers and users meet to discuss what the software should do. All requirements are gathered, analyzed, and a basic plan for the project is created.

• **User Design** – Developers create simple prototypes or models of the software. Users try them, give feedback, and suggest changes. This helps ensure the software meets user needs.

• **Construction** – The actual software is developed in this phase. Coding, improving features, and fixing issues are done using fast development tools.

• **Cutover** – The completed software is tested, different modules are integrated, and the final system is deployed for user use. User acceptance testing is also done here.

When to use the RAD Model?

- **Well-understood Requirements** – RAD is best used when the project requirements are clear and unlikely to change much.

- **Time-sensitive Projects** – It is suitable when the software must be delivered quickly within a short time.
- **Small to Medium-Sized Projects** – RAD works well for projects that are not too large and can be handled by small teams.

- **High User Involvement** – It is ideal when users can give regular feedback during development.

- **Innovation and Creativity** – RAD is useful for projects that need experimentation, new ideas, and flexible design.

- **Prototyping** – It is best when building and improving prototypes is an important part of development.

- **Low Technological Complexity** – RAD suits projects that use simple and familiar technologies.

Objectives of Rapid Application Development Model (RAD)

Fast Development

The main objective of RAD is to develop software quickly. By using rapid prototyping and short development cycles, a working system can be delivered in less time.

Flexibility and Adaptability

RAD easily supports changes in requirements. If users suggest modifications, they can be included without affecting the entire system.

Active User Involvement

Users participate throughout the development process. Their continuous feedback helps in building a system that meets real user needs.

Improved Communication

Regular interaction between developers and users ensures clarity, reduces misunderstandings, and keeps everyone aligned with project goals.

Better Quality through Prototyping

Prototypes allow early testing and validation. Errors and design issues are identified early, improving overall software quality.

Higher Customer Satisfaction

Since users are involved and working versions are delivered quickly, the final product closely matches expectations, leading to better customer satisfaction.

Advantages of RAD Model

- **Fast development:** Reusable components reduce development time.
- **Early customer feedback:** Users see prototypes early and give feedback.
- **Lower development effort:** Fewer developers are needed due to automation.
- **Better quality:** Powerful tools help build quality software quickly.
- **Easy progress tracking:** Development progress is visible in each phase.
- **Flexible to changes:** Changing requirements are easy to handle due to short cycles.
- **High productivity:** Small teams can deliver results faster.

Disadvantages of RAD Model

- **Needs skilled professionals:** Developers must be experienced with advanced tools.
- **Depends on reusable components:** Lack of reusable parts can cause failure.
- **High coordination needed:** Team leader must closely manage developers andcustomers.
- **Not suitable for all systems:** Systems that cannot be modularized cannot use RAD.
- **Continuous customer involvement required:** Lack of user commitment can fail theproject.
- **Costly for small projects:** Tool and automation costs may exceed the budget.
- **Not suitable for every application:** Some projects do not fit RAD structure.

Agile Model : The Agile Model is a software development approach that develops software in small, usable increments called iterations, with regular customer feedback and continuous improvement.

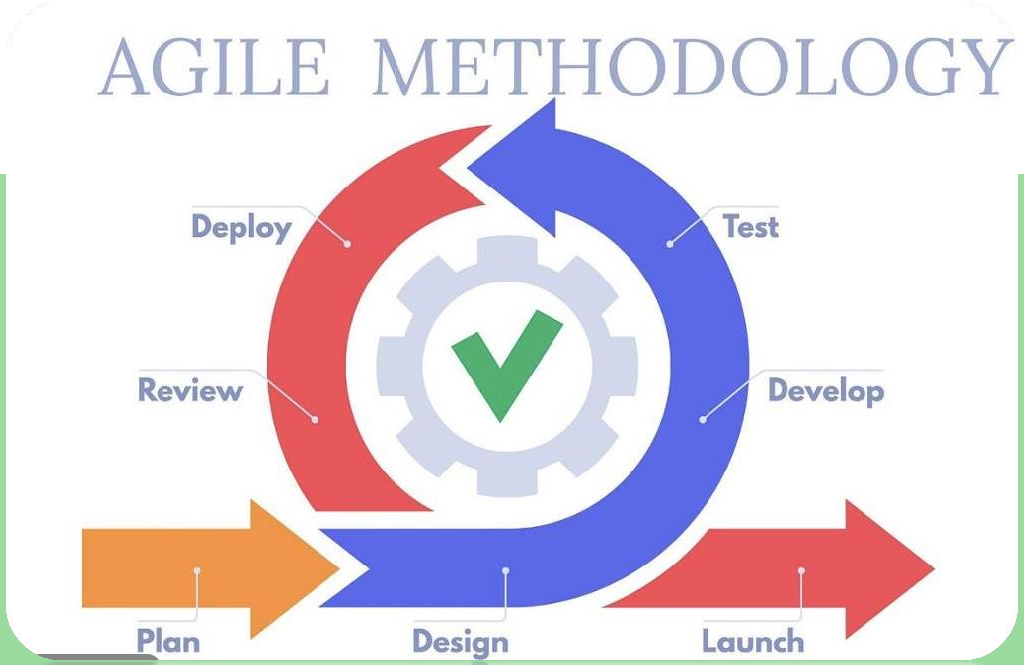
- Iterative and incremental development
- Frequent delivery of working software
- Continuous customer involvement
- Adaptive to changing requirements
- Emphasis on collaboration and communication
- Continuous testing and integration

Example :

In Agile development, instead of delivering the entire system at the end of the project, the development team delivers a functional subset of the system after each iteration (called a sprint), evaluates it with stakeholders, and refines the system based on feedback.

Agile Iterative Cycle (Sprint Cycle) :

- Plan → Design → Develop → Test → Review → Repeat
- Software improves step-by-step, not all at once



Evolution of Software : Evolution of Software refers to the **continuous change, improvement, and adaptation of software after it has been developed and deployed.**

Software does not remain fixed. It is **updated, modified, and enhanced** over time to meet **new user requirements, technology changes, and security needs.**

Why Software Evolves :

Software must evolve because:

- User requirements change
- New technologies appear
- Security vulnerabilities are discovered
- Performance improvements are needed
- Business processes change

Example

- **WhatsApp** initially allowed only text messaging
- Later features added:
  - Voice calling
  - Video calling
  - Status updates
  - Payment system

  This continuous improvement is called **software evolution**.

Stages in the Evolution of Software

Software development is divided into **four major eras**.

1. Early Era (1950–1960)

Characteristics

- Limited distribution
- Custom software
- Batch processing systems

During this era, software was developed **for specific hardware and organizations**. Programs were not widely shared.

Example

Banking systems or scientific programs developed **for a single computer**

2. Second Era (1960–1972)

Characteristics

- Multi-user systems
- Database systems
- Real-time software
- Multiprogramming
- Product software

Computers became more powerful and interactive systems started to appear.

Example

- Airline reservation systems
- Early database management systems

3. Third Era (1973–1985)

Characteristics

- Embedded intelligence
- Consumer software
- Distributed systems
- Low-cost hardware

Computers became cheaper and software started reaching **common users**.

Example

- Personal computer software
- Early operating systems like **MS-DOS**

4. Fourth Era (1987 – Present)

Characteristics

- Powerful desktop systems
- Expert systems
- Artificial Intelligence
- Network computers
- Parallel computing
- Object-Oriented Technology

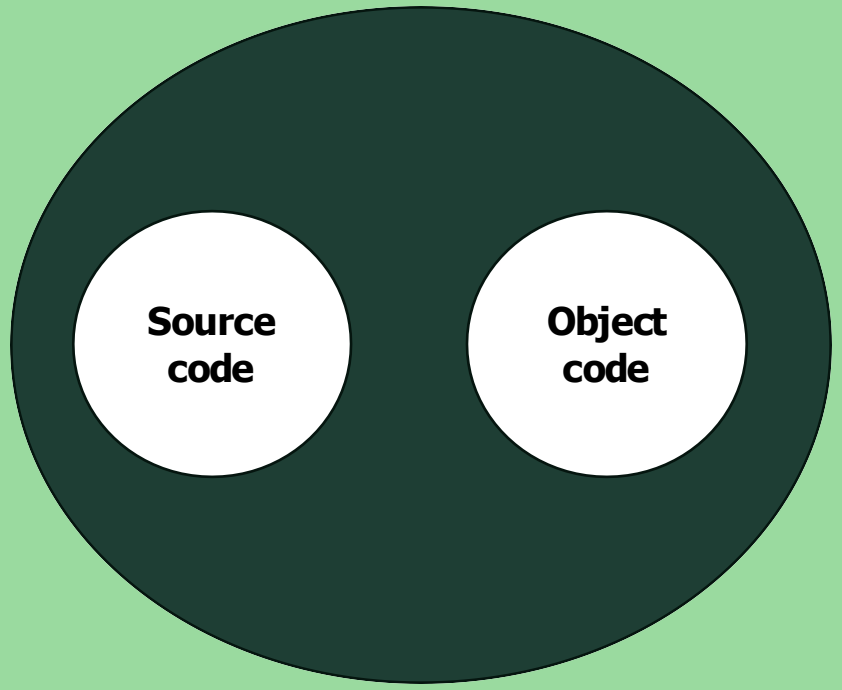
Software became more advanced and intelligent.

Example

- Artificial Intelligence applications
- Cloud computing systems
- Machine learning software

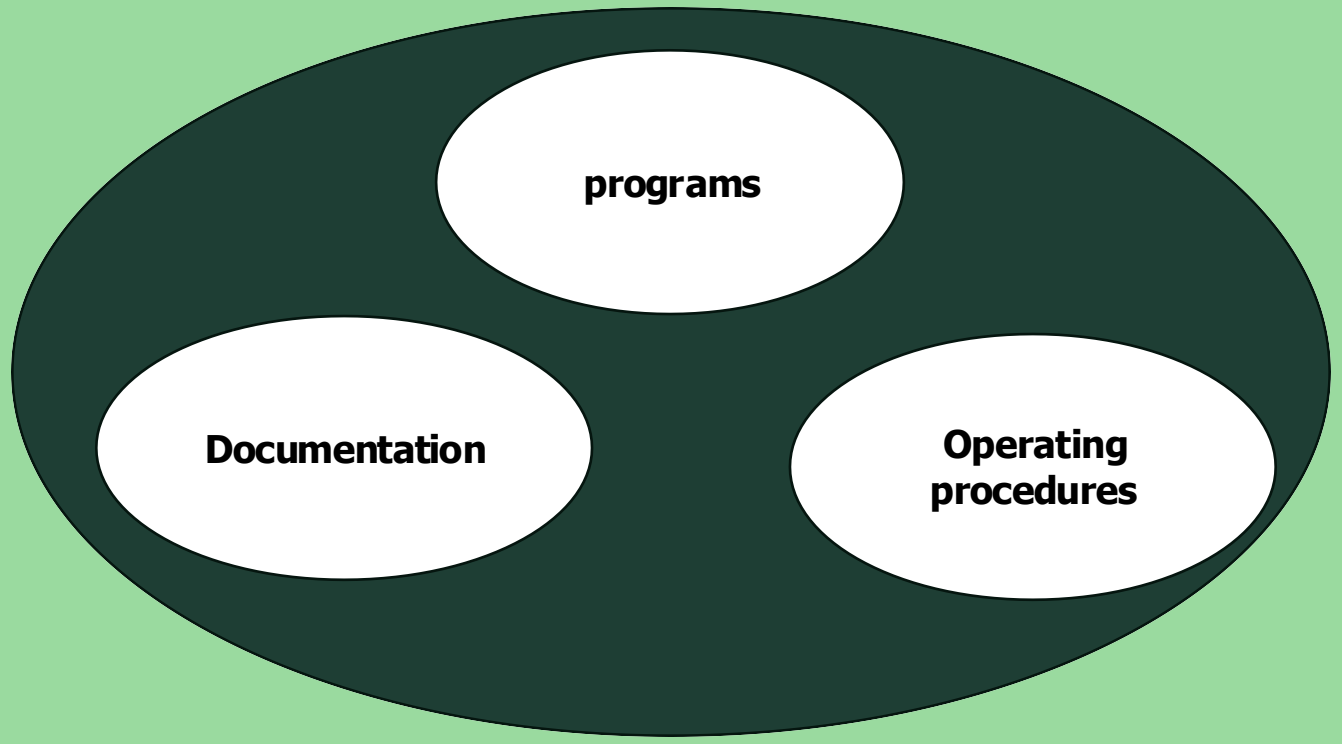
PROGRAM AND SOFTWARE PRODUCT

A Program is a subset of software and it becomes software only of documentation and operating procedure manuals are prepared. Program is a combination of source code and object code.



PROGRAM = SOURCE CODE +OBJECT CODE

Software Products : A software product consists not only of the program code but also of all the associated document such as the requirements specifications documents the design documents the test document the operating procedures which includes user manuals and operational manuals.



SOFTWARE= PROGRAM +DOCUMENTATION +OPERATING PROCEDURES

Generic View of a Software Engineer :

Generic View of Software Engineering refers to the **overall framework or common activities that software engineers follow to develop and maintain software systems.**

It describes the **general tasks, processes, and responsibilities** involved in software development.

1.Framework Activities (Main Activities)

These are the **core steps followed in every software project.**

Activity	Meaning	Example
Communication	Discuss requirements with stakeholders	Meeting with hospital staff before creating Hospital Management System
Planning	Decide project schedule, cost, and resources	Planning timeline for developing a mobile app
Modeling	Design system architecture and diagrams	Creating UML diagrams for a banking system
Construction	Coding and testing the software	Writing Java or PHP code and performing testing
Deployment	Deliver software to users and maintain it	Launching a website or application

2.Umbrella Activities in Software Engineering :

Umbrella activities support **all stages of software development**.

Examples:

- **Software Quality Assurance (SQA)** – Ensuring software quality
- **Risk Management** – Handling possible problems
- **Configuration Management** – Managing versions of software
- **Measurement** – Evaluating performance
- **Documentation** – Maintaining project documents

These activities run **throughout the entire software process**.

Software Development Life Cycle (SDLC) : The Software Development Life Cycle (SDLC) is a structured framework used by software organizations to design, develop, and test high-quality software. It defines the entire process of software production, from the initial idea to the final deployment and maintenance.

The goal of the SDLC is to produce high-quality software that meets or exceeds customer expectations, reaches completion within time and cost estimates, and is efficient to maintain.

Why is SDLC Important?

Without a structured SDLC, software development becomes chaotic. Teams may miss requirements, blow budgets, or deliver buggy code that doesn't solve the user's problem.

SDLC provides:

- **Visibility:** Stakeholders know exactly what is happening at every stage.
- **Quality Control:** Testing is integrated into the process, not an afterthought.
- **Risk Management:** Potential pitfalls are identified during the planning phase.
- **Cost Estimation:** Helps in predicting timelines and budgets accurately.

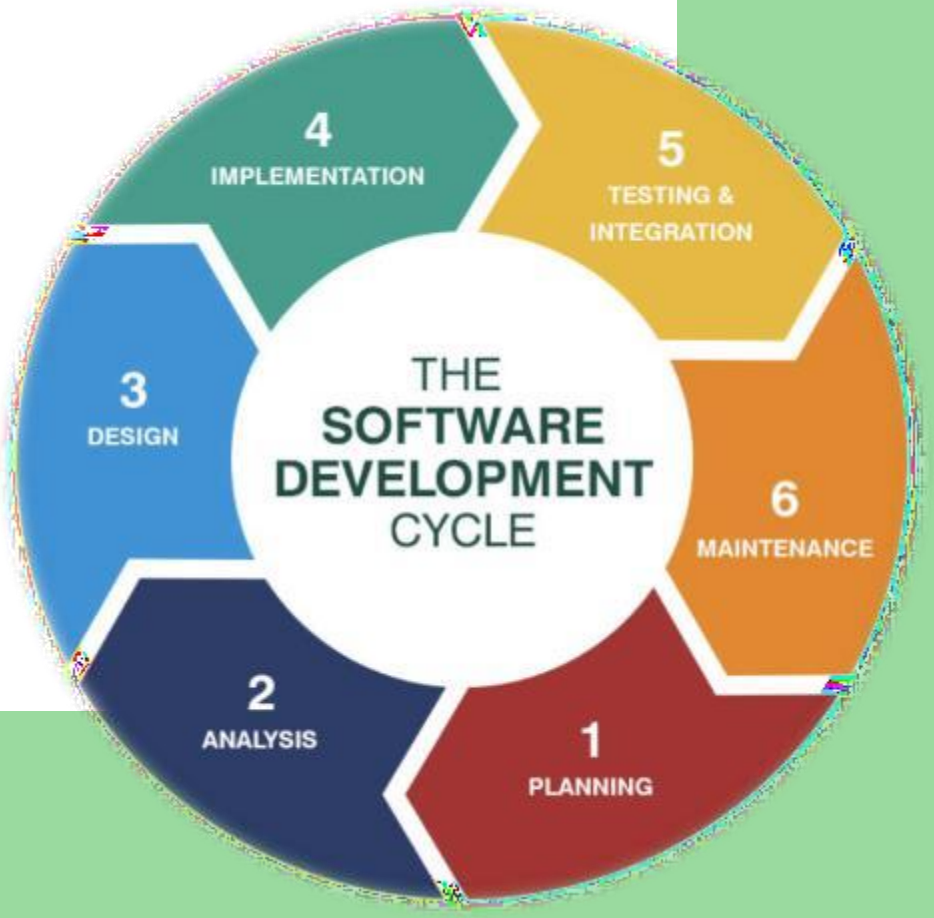
1.Planning : Planning is the process of deciding what to build,
why to build it, how much time it will take, and how much it will cost.

What happens here

- Identify problem
- Check feasibility (Can we build it?)
- Budget estimation
- Team selection

Example:
College wants a Student Portal → team decides features: login, marks, attendance.

Output: Project Plan Document



2.Analysis (Requirement Analysis) :

Requirement analysis is the process of collecting, understanding, and documenting user needs.

What happens :

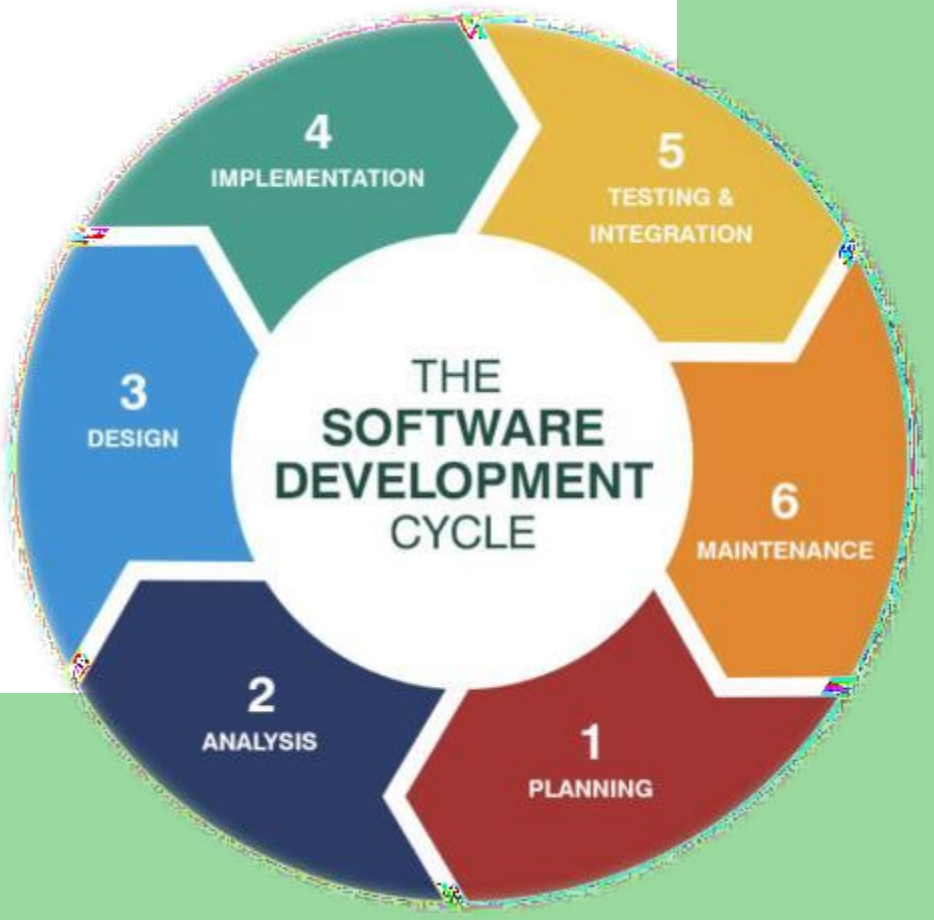
- Meet users (dient / students / staff)
- Ask questions
- Write SRS (Software Requirement Specification)

Example

Should students download marks?

- Should teachers upload assignments?
- Should parents see attendance?

Output: SRS Document



3.Design : Design is the process of converting requirements into a technical blueprint of the system.

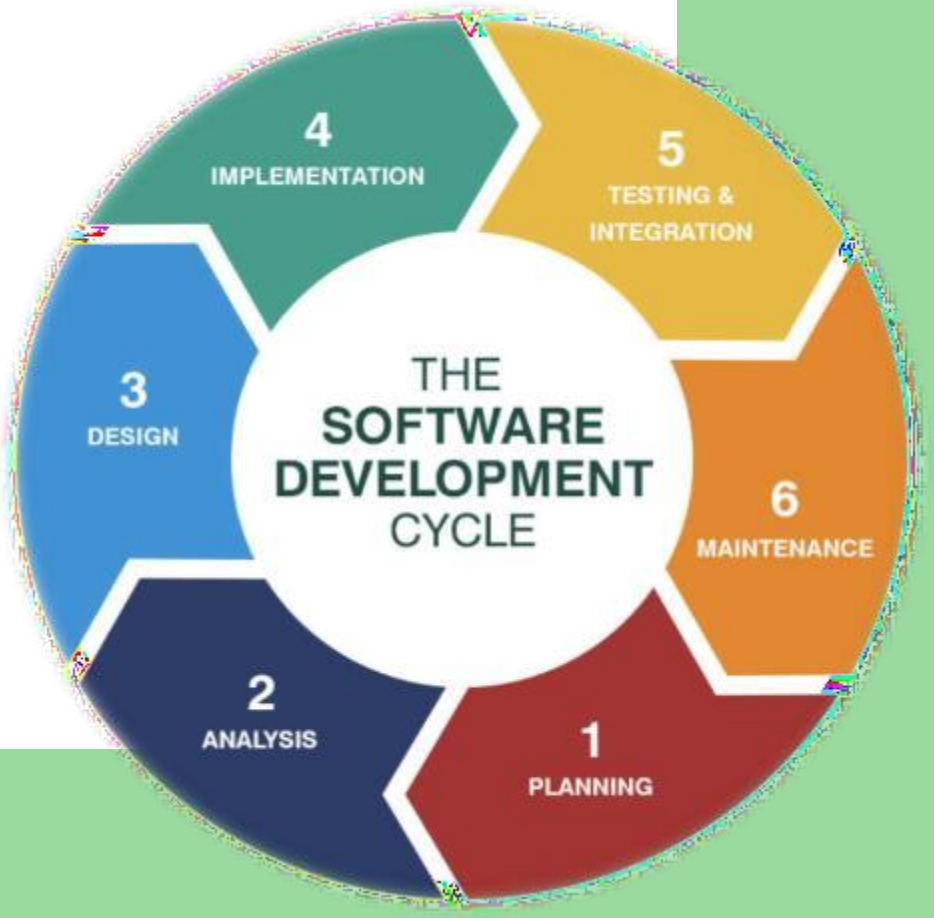
What happens :

- Database tables design
- Interface layout (UI)
- System architecture
- Flowcharts & diagrams

Example :

- Login page design
- Database table: Student(id, name, marks)
- Decide website vs mobile app

Output: Design Document / Architecture



4.Implementation (Coding) :

Implementation is the phase where programmers write the actual code based on design.

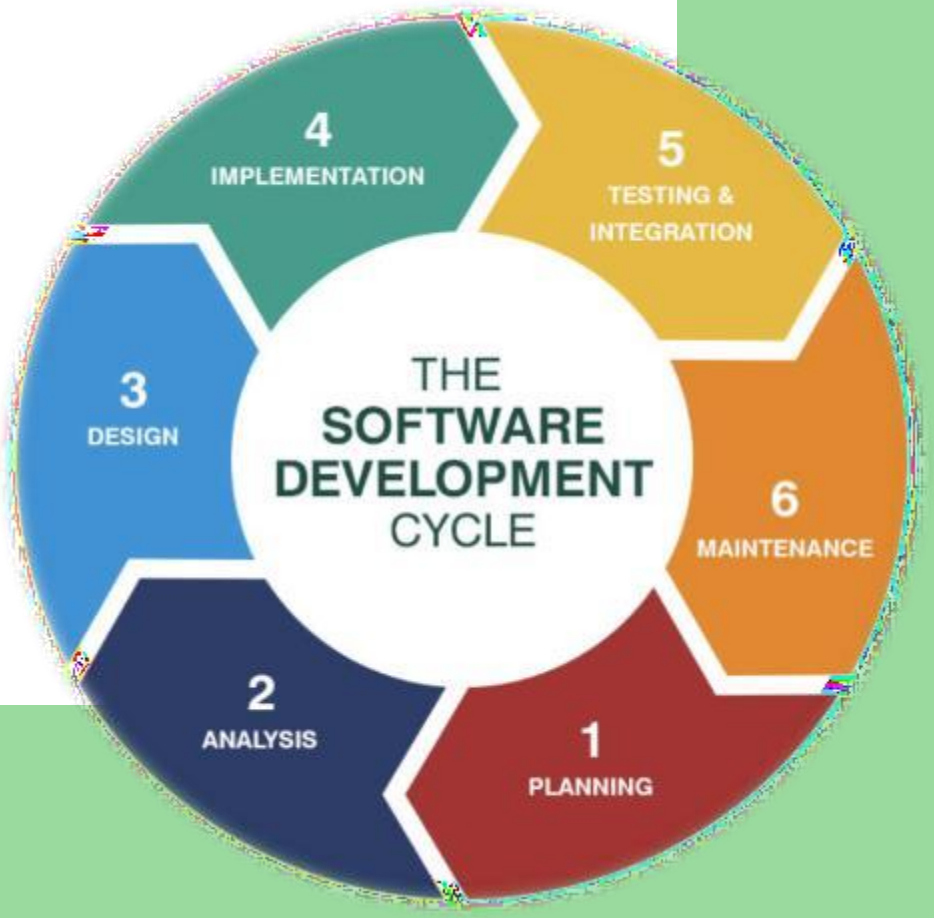
What happens :

- Developers write programs
- Use languages (PHP, Java, Python)
- Build modules

Example

- Login.php
- Upload_marks.php
- Database connection

Output: Working Software (but not tested)



5.Testing & Integration : Testing is the process of finding and fixing errors (bugs) in the software.

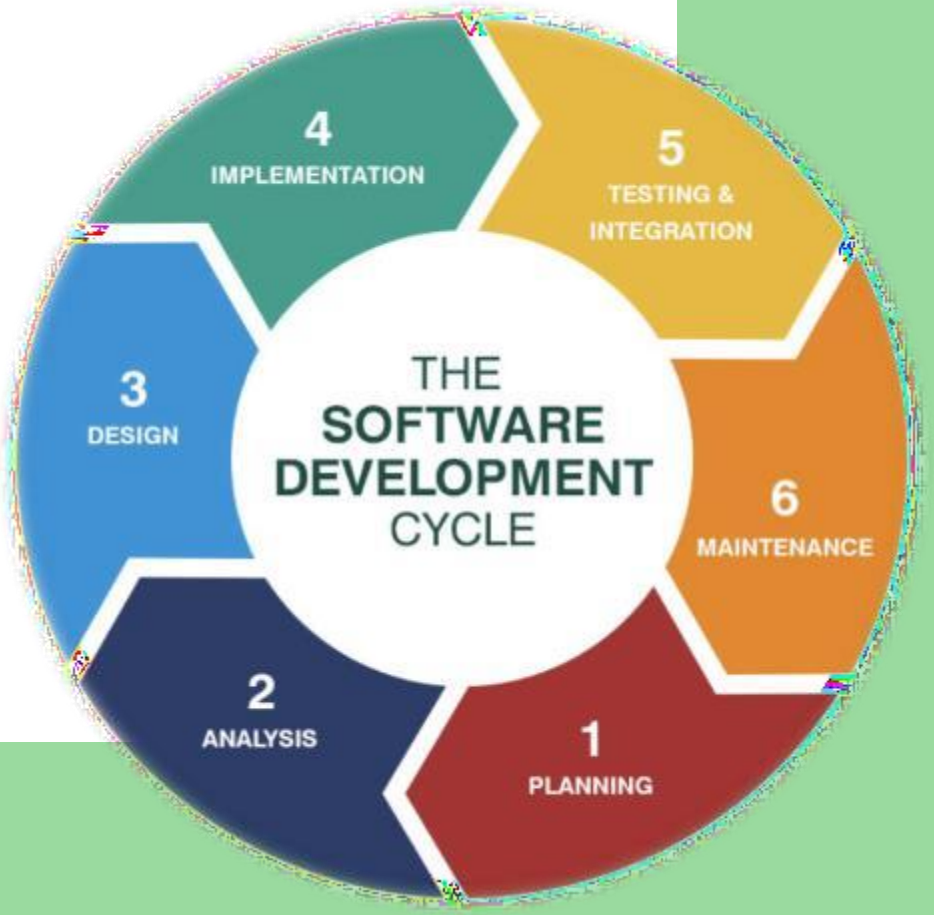
What happens :

- Unit testing
- Integration testing
- System testing
- Fix errors

Example Bugs :

- Wrong password still login
- Marks showing negative
- Page crash

Output: Bug-free Software



6.Maintenance : Maintenance is the process of updating and improving software after delivery.

What happens :

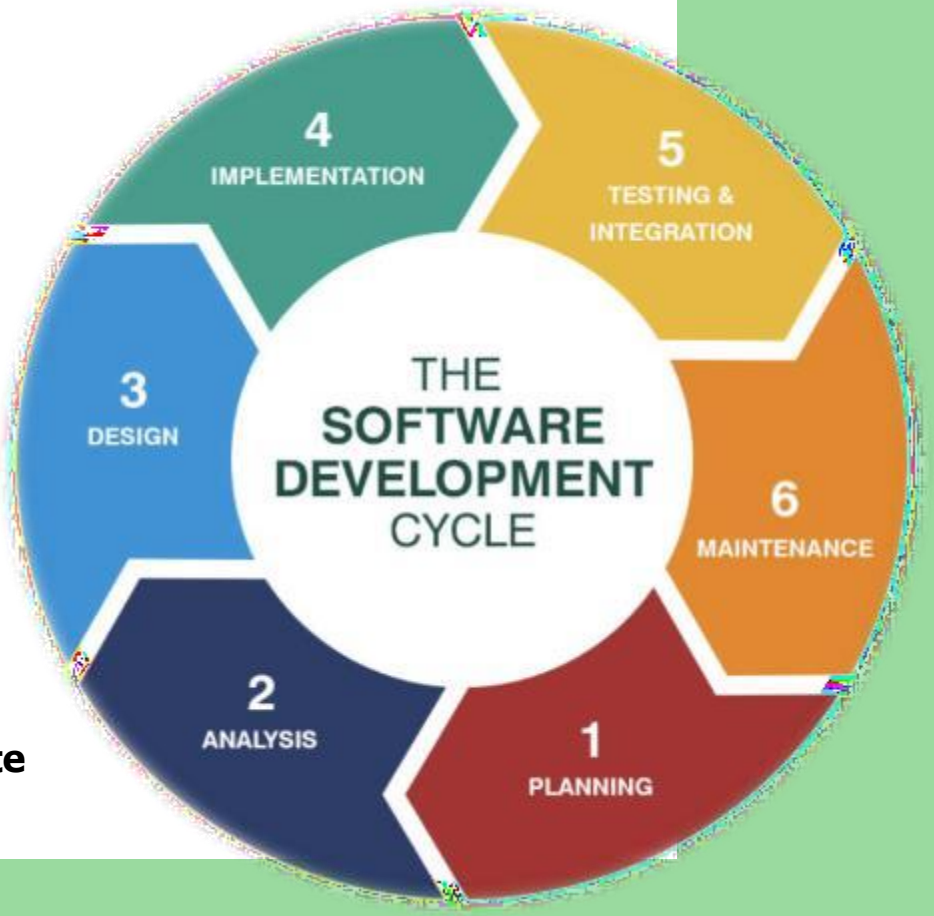
- Add new features
- Fix new bugs
- Improve speed
- Adapt to new requirements

Example

- Add dark mode
- Add online fee payment
- Update for new syllabus

Output: Updated Version (v2, v3, v4 …)

Plan → Ask → Design → Code → Test → Update



UNIT - 2 REQUIREMENT ANALYSIS

Introduction of Requirement :

Requirements are defined as descriptions and specifications of a system. It may range from a high – level abstract statement of a services or of a system constraint to a detailed mathematical functional specification.

System requirement may be either functional or non-functional requirements.
Requirement are classified under user requirement , system requirement and domain requirement.

Requirements Analysis

Requirements Analysis is the process of **collecting, analyzing and documenting user needs** for a software system.

- Understand user needs
- Define system functions
- Remove ambiguity (ambiguity means **the requirement is not clear)**
- Prepare **SRS document**

Types of Requirements :

1. Functional Requirements

These describe **what the system should do**.

Examples:

- User login
- Online payment
- Report generation

2. Non-Functional Requirements

These describe **how the system should perform**.

Examples:

- Security
- Performance
- Reliability

Types of System Requirements

1. User Requirements :

- Written for **end users**
- Simple language

Example

"User should be able to book a ticket online."

2. System Requirements :

- Detailed technical description
- Used by developers

3. Domain Requirements :

- Requirements related to **specific industry**

Example

Hospital system must follow **medical regulations**

Requirements Engineering Process :

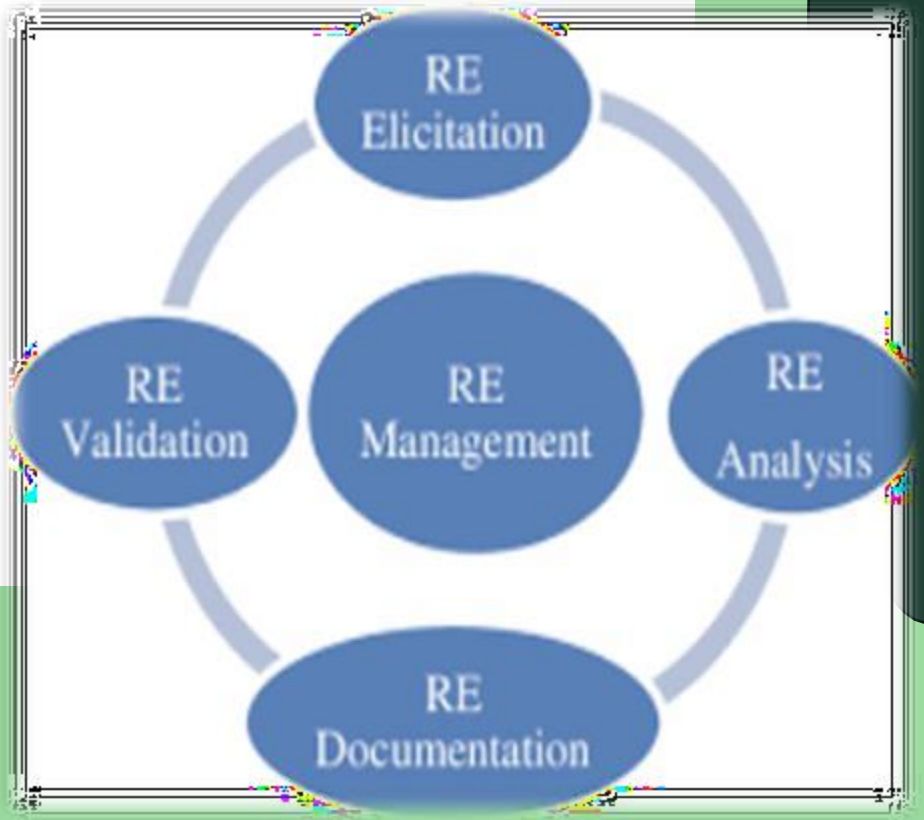
Requirements Engineering is the **process of discovering, analyzing, documenting, and maintaining the requirements of a software system.**

Or,

Requirements engineering is the process of establishing the services that a system should provide and the constraints under which it must operate.

Steps involved in requirement development.

1. Requirement Elicitation
2. Requirement Analysis
3. Requirement Specification
4. Requirement Validation
5. Requirement Management



1.Requirement Elicitation :

Requirement elicitation involves collecting requirements from stakeholders using interviews, questionnaires, and observations.

- Interviews
- Questionnaires
- Meetings
- Observation
- Prototyping

Real Example

Hospital Management System

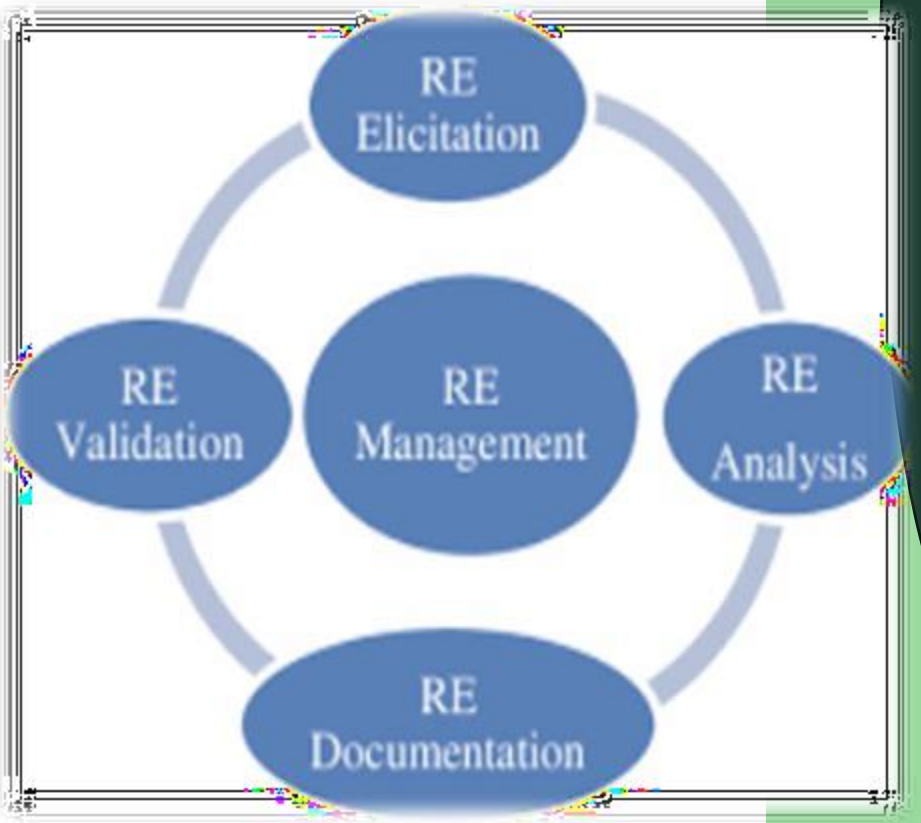
Developers meet with:

- Doctors
- Receptionists
- Hospital managers

They ask questions like:

- How are patient records stored?
- How are appointments booked?
- What reports are required?

From this discussion, requirements are collected.



2.Requirement Analysis :

Requirement analysis is concerned with refining and structuring the requirements.

Activities

- Remove unclear requirements
- Identify conflicts
- Prioritize requirements
- Check feasibility

Real Example

In the **hospital system**:

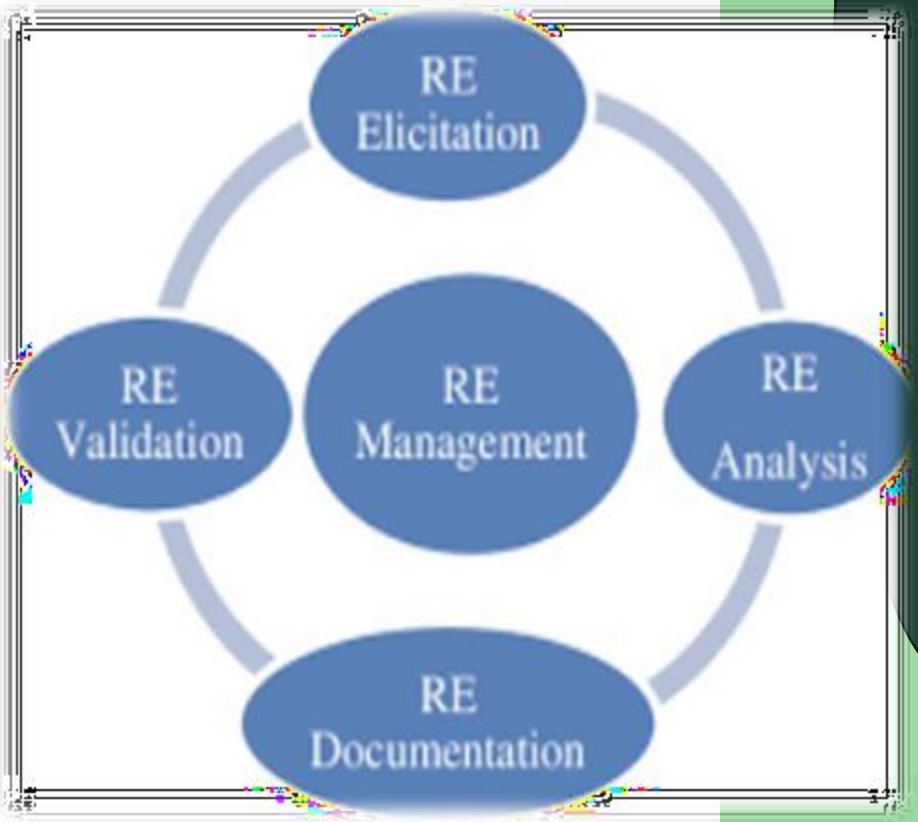
Collected requirement:

"Patient records should be stored safely."

Analysis result:

Developers define it more clearly:

- Records stored in database
- Only doctors can access them
- Data must be encrypted



3.Requirement Specification :

SRS is a complete description of the behavior and requirements of a software system.

Contents of SRS

- Functional requirements
- Non-functional requirements
- System interfaces
- Constraints

Real Example

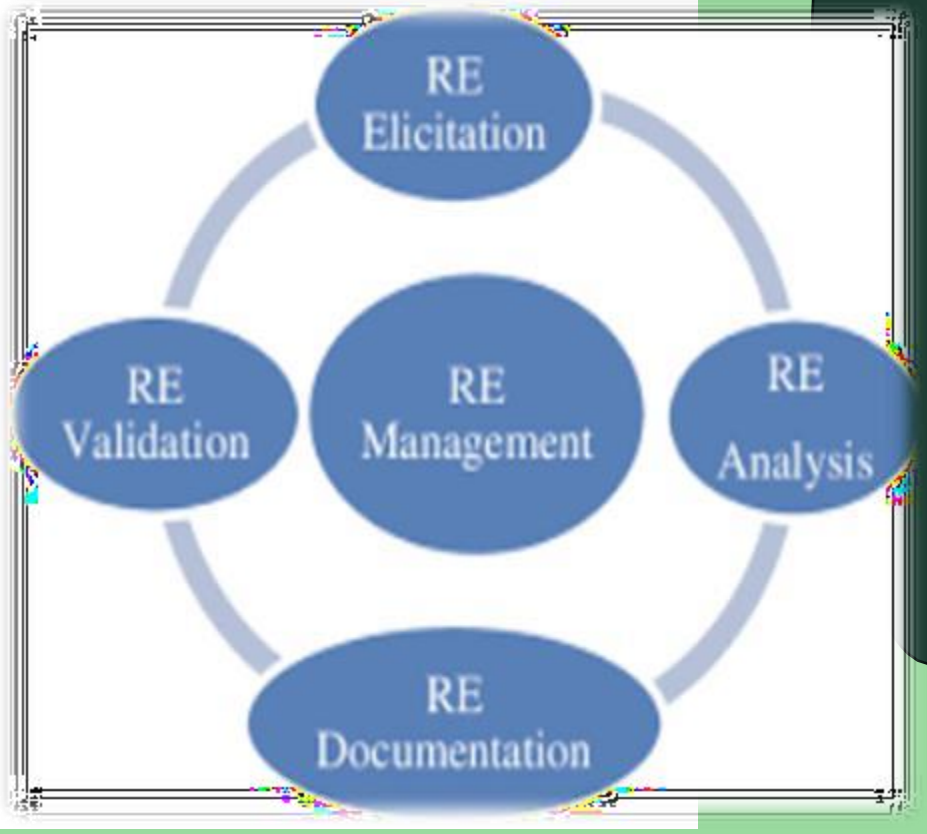
For **Online Shopping System**, the SRS may include:

Functional requirements:

- User registration
- Product search
- Online payment

Non-functional requirements:

- Website loads within 3 seconds
- Secure payment system



4.Requirement Validation :

Validation checks that the requirements define the system that the customer really wants.

Methods

- Requirement reviews
- Prototyping
- Testing scenarios
- Stakeholder meetings

Real Example

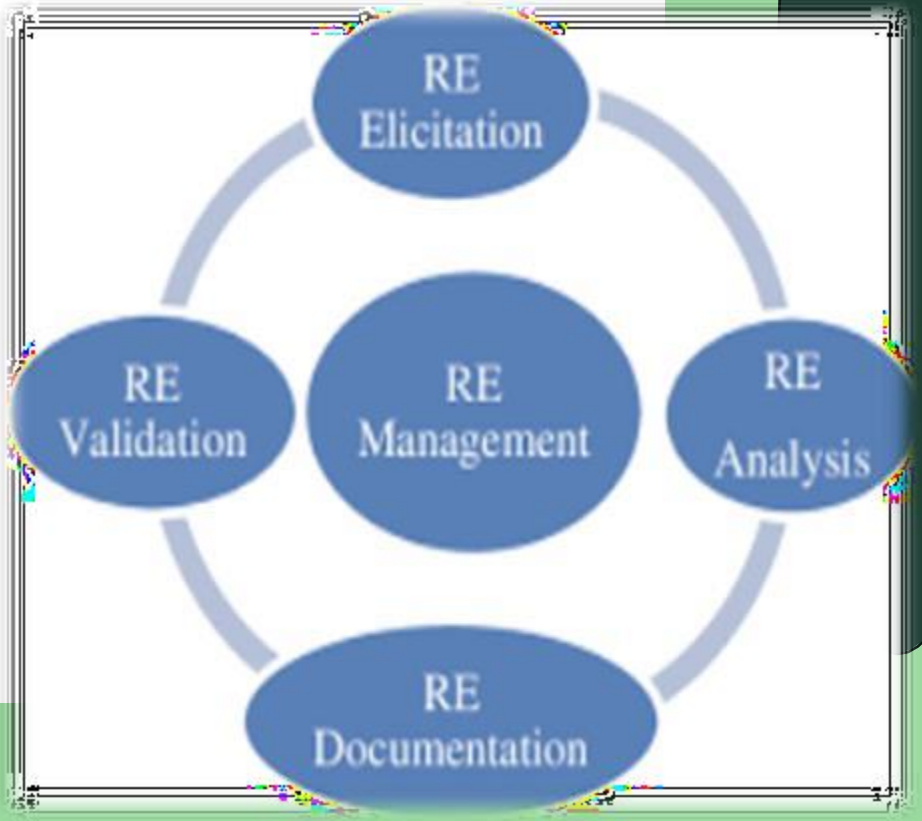
For **Banking Software**:

Developers show the SRS to bank officials.

Bank officials check:

- Are ATM services included?
- Are online transactions supported?
- Is security defined?

If something is missing, it is corrected.



5.Requirement Management :

Requirement management deals with controlling changes to requirements.

Activities

- Track requirement changes
- Update documentation
- Communicate changes to developers

Real Example

In **Food Delivery App** development.

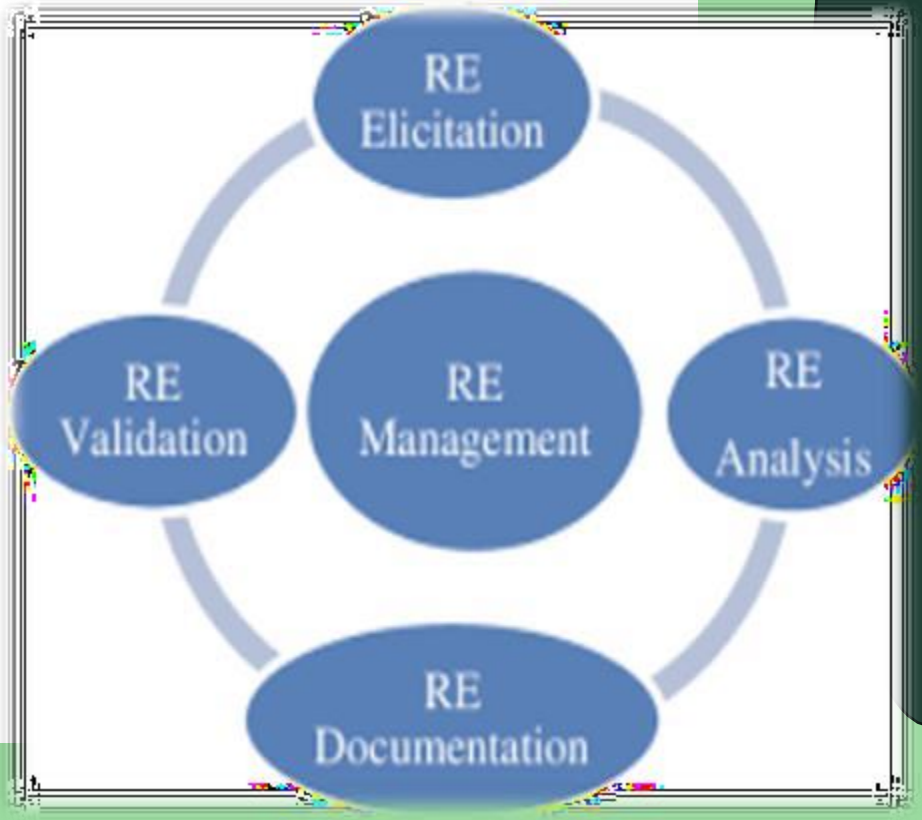
Initial requirement:

- Order food online

Later new requirement added:

- Live delivery tracking

Developers update the system requirements and modify the software accordingly.



What Is an SRS Document?

An SRS document acts as a guide for the development team, detailing how the software should perform and interact. It outlines the purpose, behavior, and functionality of a software application.

Think of an SRS as a blueprint or roadmap for the software you're going to build.

The elements that comprise it should include:

- **Purpose:** Why the software is being built.
- **Description:** What the software will do.
- **Requirements:** Specific functionalities and constraints.
- **Links:** Connecting user stories to specific requirements.
- **Approval Process:** Ensuring all stakeholders are aligned.

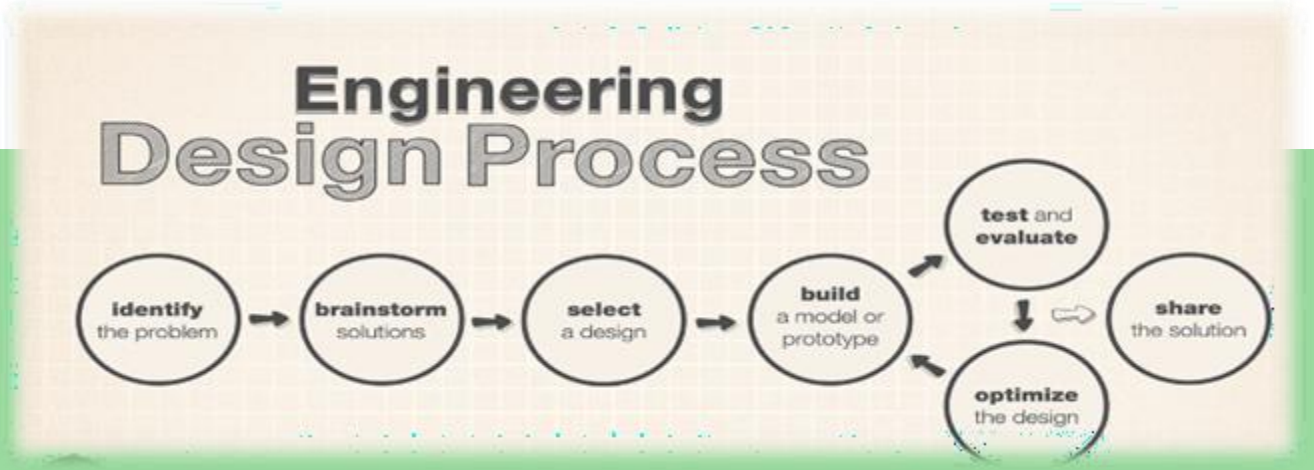
UNIT - 3 DESIGN CONCEPTS AND PRINCIPLES

Design Process

The design process transforms requirements into a blueprint for construction.

- Requirement Analysis
- System Design
- Detailed Design
- Implementation
- Testing

Example: ATM system design → input (card, PIN) → processing → output (cash).



Design Concepts

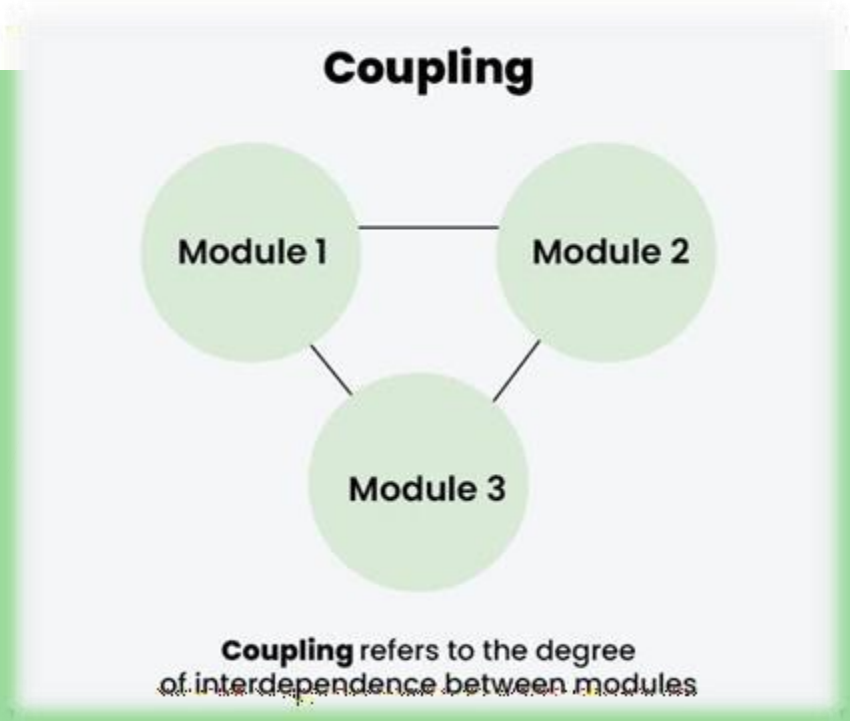
- Abstraction
- Refinement
- Modularity
- Software Architecture
- Control Hierarchy
- Data Structure

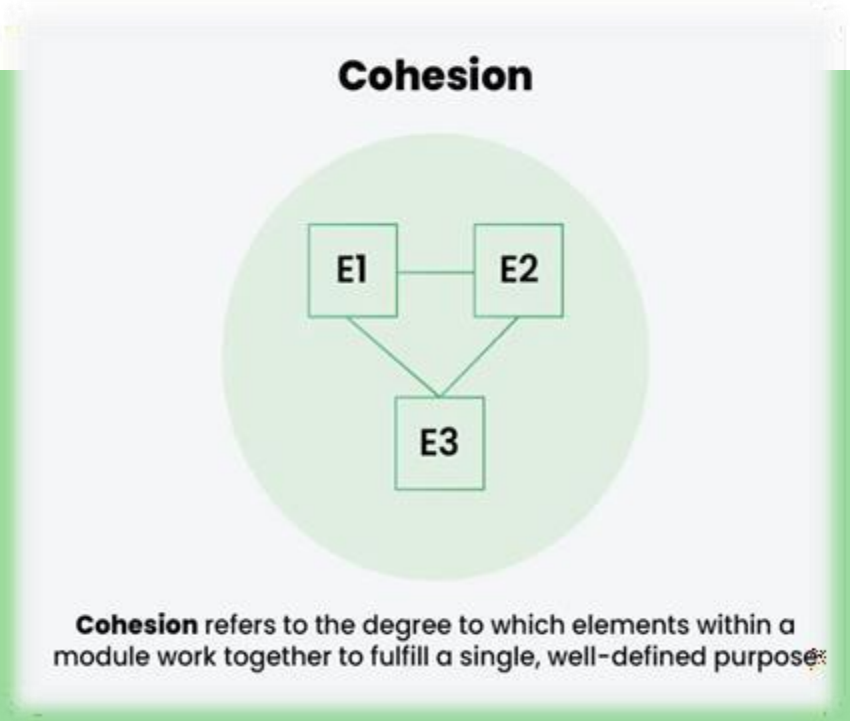
Example: Library system → modules like Book, User, Issue, Return.

Design Principles

- Avoid redundancy
- Keep it simple (KISS)
- Separation of concerns
- High cohesion & low coupling

Example: Login module separate from payment module.





Effective Modular Design :

Effective modular design is a design approach in which a software system is divided into well-defined, independent modules with high cohesion and low coupling, making the system easier to understand, develop, test, and maintain.

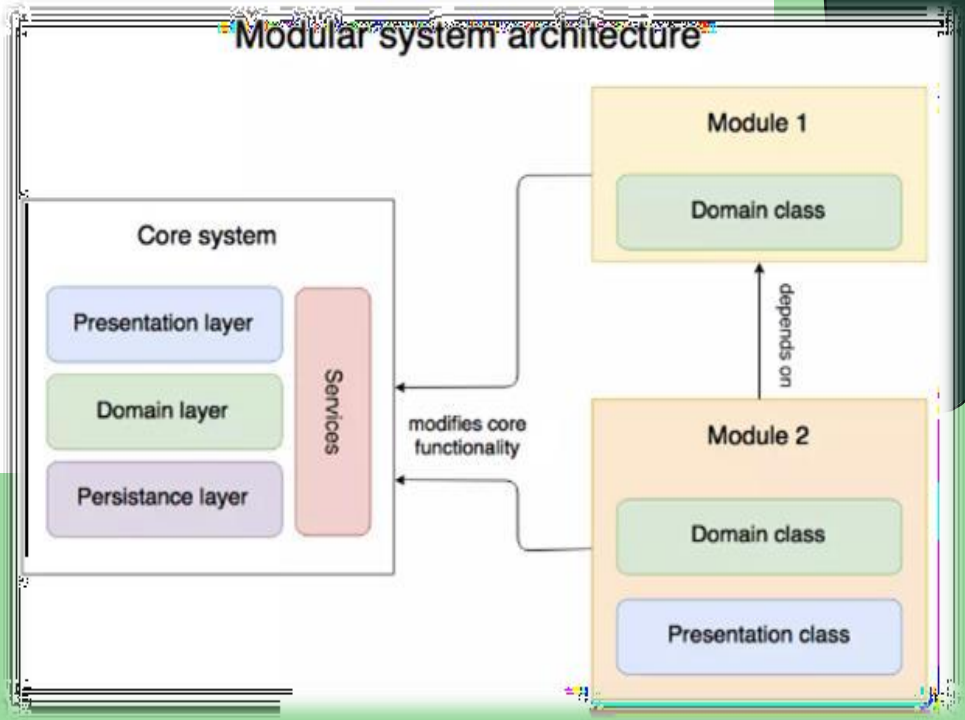
Characteristics:

- Small modules
- Independent functionality
- Easy to maintain

Advantages:

- Reusability
- Easy debugging

Example: E-commerce: Cart module, Payment module, Product module.



Human Computer Interface (HCI) : Human Computer Interface (HCI) is the study and design of the interaction between users (humans) and computer systems, focusing on usability, efficiency, and user experience.

Goals:

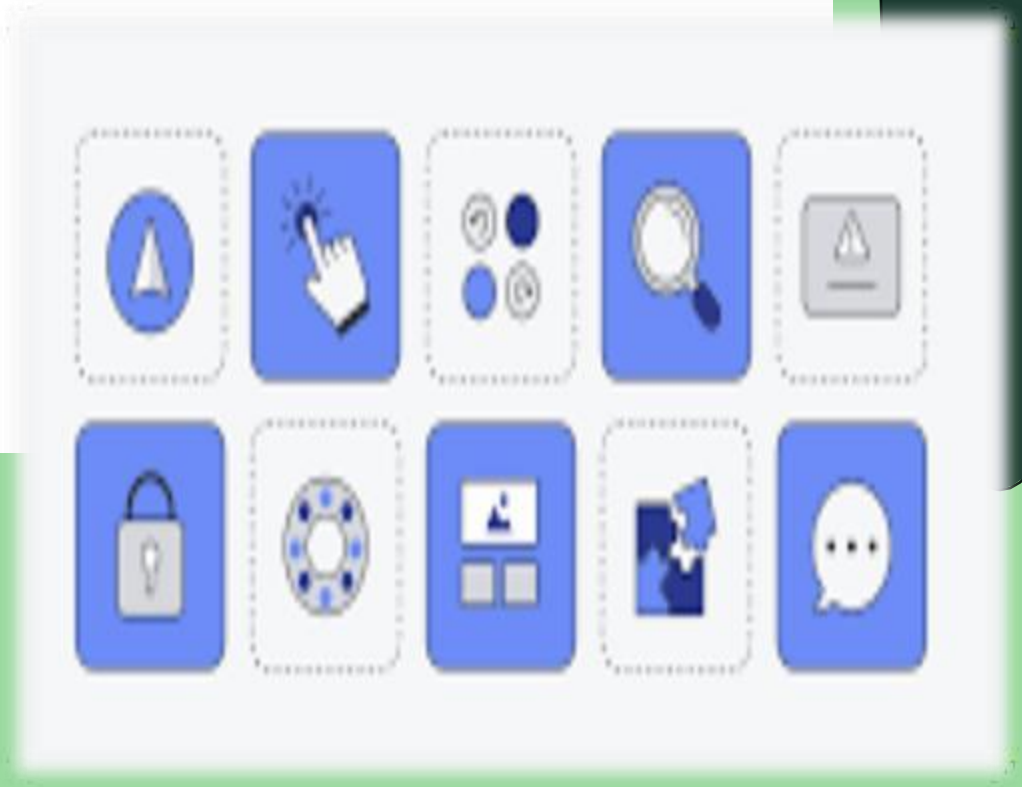
- Usability
- Accessibility
- User satisfaction

Example: Mobile app UI design.

Interface Design Guidelines :Interface design guidelines are a set of principles and recommendations that help in designing user interfaces that are easy to use, consistent, efficient, and user-friendly.

- Consistency
- Feedback
- Simplicity
- Error prevention
- User control

Example: Undo button in applications.



UNIT - 4 SOFTWARE QUALITY ASSURANCE & TESTING

Software Quality Assurance (SQA)

Software Quality Assurance is a set of activities that ensures the software processes and products conform to specified requirements and standards.

Quality Concepts (Important Points)

1. Correctness
2. Reliability
3. Efficiency
4. Usability
5. Maintainability

Software Testing Fundamentals : Software Testing is the process of executing a program to find errors.

Testing Goals

Errors find
Quality improve
User satisfaction

Types of Testing

White Box Testing
Black Box Testing
Basic Path Testing

Testing Strategies :

A systematic plan to perform testing from unit level to system level.

Testing Levels

Unit Testing
Integration Testing
Validation Testing
System Testing

Full Flow

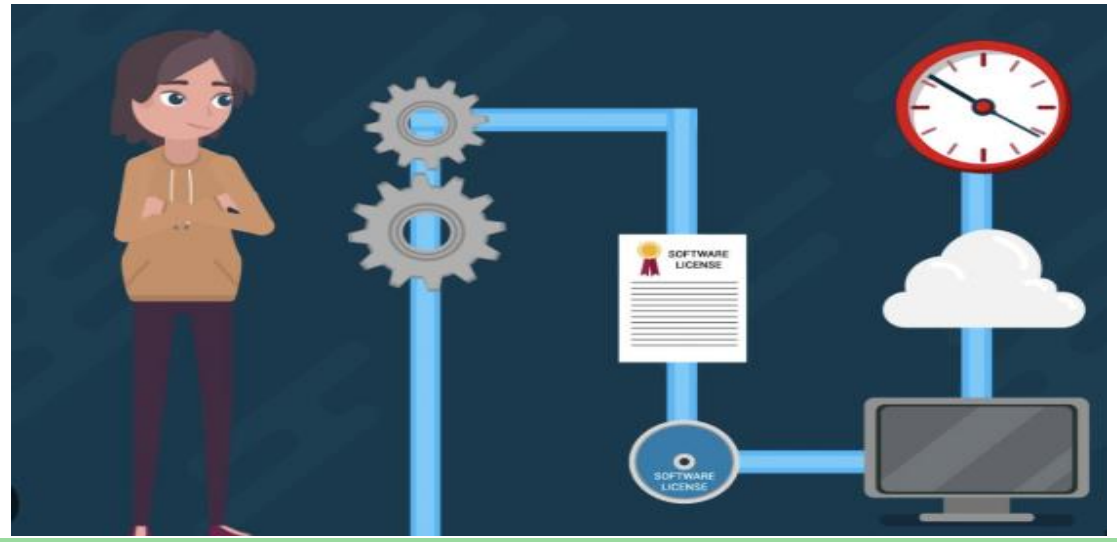
Unit → Integration → Validation → System

UNIT - 5 SOFTWARE REUSE, MAINTENANCE, SCM & CASE

1. Software Reuse

Software reuse is the process of creating software systems from existing software rather than building them from the ground up.

Reuse Process: Involves identifying assets (code, designs, documentation), assessing their fluid integration, and adapting them to the new system requirements.



Software Reuse Process :

1.Identify Components

Find which components can be reused.
Identify existing modules or code that can be used again.

2.Classify Components

Organize components into categories
Group components based on their function, application, or data.

3.Store in Repository

Save components in a storage.
Store reusable components in a repository for future use.

4.Retrieve Components

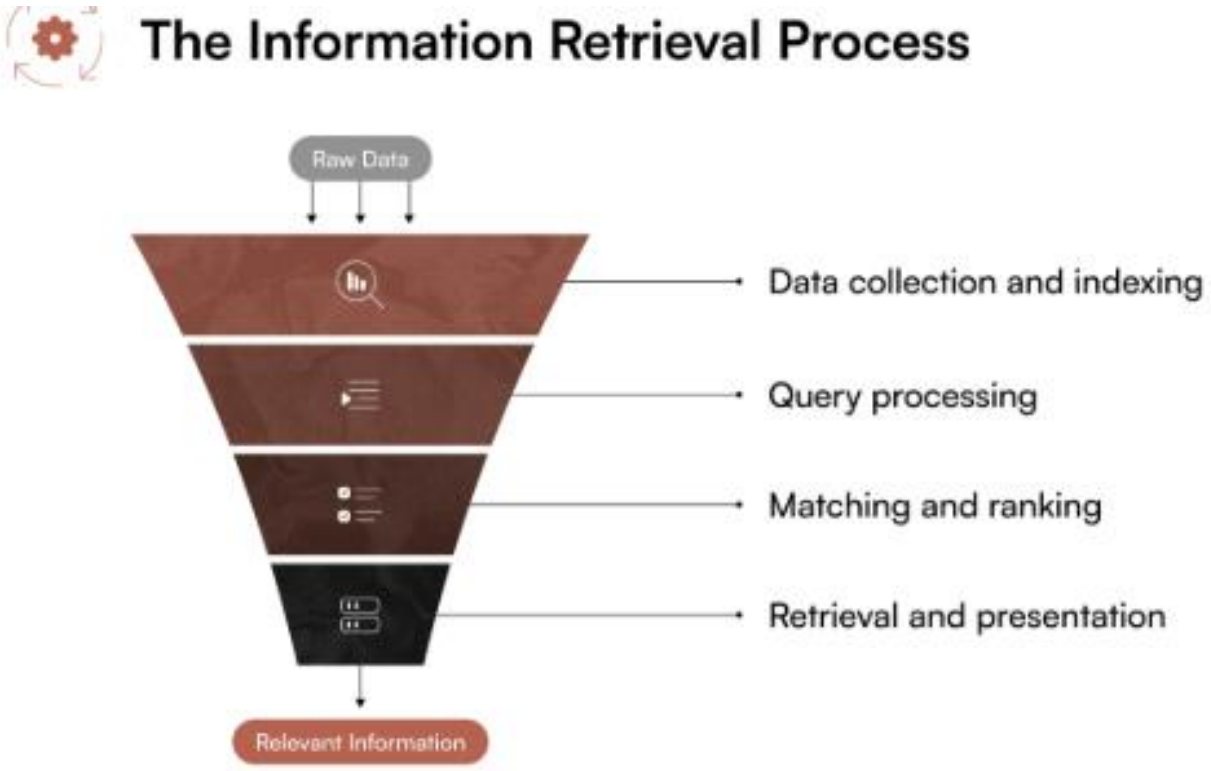
Get components when needed.
Search and retrieve the required components from the repository.

5.Integrate into New System

Use in new software
Integrate the selected components into the new system.

Retrieving Components :

Retrieving components is the process of searching and selecting reusable software components from a repository for use in a new system.



Economics of Software Reuse : Economics of software reuse refers to the cost and benefits involved in reusing software components instead of developing them from scratch.

Advantages of Software Reuse :

1. Time Saving
2. Cost Reduction
3. Better Quality

Disadvantages of Software Reuse :

1. Difficult to Find
2. Compatibility Issues

Software Maintenance :

Software maintenance is the process of modifying software after delivery to correct faults, improve performance, or adapt to changes.

Need for Maintenance

- Bugs fix
- Add New features
- Performance improve
- Environment change

Types of Maintenance

1. Corrective : Errors fix
2. Adaptive : Environment change
3. Perfective : Improve performance
4. Preventive : Future problems avoid

Software Configuration Management (SCM)